ARCH Portal

Development Guide

Johns Hopkins University

IT@JHU Research Computing

Advanced Research Computing at Hopkins (ARCH)

Table of Contents

ARCH Portal (CLOACK Stack) 3
- High-level design (draft) - Authentication + Scheduler stack 3
- start.sh -- Stack Lifecycle Management 4
- Docker Images 9
- Billing & Invoicing System 10
- Populating Synthetic Data for Testing 18
- 10. Slurm CLI Utilities 22
- Testing 26
- User Guide 27
- Multi-Cluster Architecture 28

Features 32
- Branch Overview 32
- Documentation Index 32
- Feature Comparison: main vs prod vs dev deployment 32
- What prod has over main (69 commits) 36
- What dev has over prod (259 commits) 36
- Billing-Free Resources 36
- Slurm Account Naming Convention 36
- UID/GID Ranges 37

Staff Help & Reference Guide 38
- {{ section.title }} 38

ColdFront 39
- Build 39
- Startup (entrypoint.sh) 39
- Directory Structure 39
- Key Config 39
- Environment Variables 40
- Port 40
- Development 40
- Admin Features 40

Custom Configurations 42
- Structure 42
- How These Customizations Work 42
- Key Customizations 43
- Deployment Notes 45

Billing Setup Guide 47
- Step 1: Import Departments 47
- Step 2: Assign PIs to Departments 47
- Step 3: Create Billing Rates 47
- Step 4: Configure Allocation Billing 48

ARCH Portal (CLOACK Stack)

The ARCH Portal is built on the CLOACK Stack, a containerized identity and resource management framework for HPC clusters at Johns Hopkins University. It integrates ColdFront, OpenLDAP, Keycloak, and Slurm into a container-based platform for user provisioning, job scheduling, and billing.

Component	Role
Coldfront	HPC resource allocation portal -- orchestrates LDAP/Slurm sync, home dirs, PI scratch links
LDAP / Open**L**DAP	Central directory -- users, groups, PIs across JHU and Schmidt (SSCI) domains
OpenLDAP	Single OpenLDAP instance with three naming contexts (`dc=arch`, `ou=jhu`, `ou=schmidt`)
Auth (OIDC/SSSD)	OIDC via Keycloak + SSSD for PAM/NSS resolution inside containers
Cluster/**C**ontainers	Slurm HPC cluster + Docker Compose orchestration -- builds, networks, and manages all services as a unified stack
Keycloak	Identity broker -- per-realm LDAP federation, Entra ID integration, token issuance

```
Deployed on the ARCH cluster at Johns Hopkins University, supporting both JHU and Schmidt Sciences (SSCI) r...
```

```
Interactive Architecture Diagram -- click any component for details.
```

High-level design (draft) - Authentication + Scheduler stack

We are using Coldfront as Orchestrator. It making LDAP and Slurm synchronization, as well user's home directory and softlink point to PI scratch folder where they are members.

Use three separate LDAP naming contexts (suffixes) and disjoint uid/gid ranges; create cross-domain admin/group entries (DN-based) that include members from any suffix; configure Open Source Identity and Access Management "Keycloak" realms so each realm points to the correct subtree(s) or adds multiple LDAP providers.

Three suffixes on one OpenLDAP instance:

- Domain A (arch.cluster): ARCH Admins suffix **dc=arch,dc=cluster** UID/GID range starting 1000
- Domain B (jhu.arch.cluster): suffix **ou=jhu,dc=arch,dc=cluster** UID/GID range starting 10000
- Domain C (schmidt.arch.cluster): suffix **ou=schmidt,dc=arch,dc=cluster** UID/GID range starting 30000

Keep UID/GID ranges non-overlapping.

ColdFront as Directory Ground Truth: UID and GID allocation is centralized in the ColdFront database via **Affiliation.next_uid** / **next_gid** counters (atomic, race-condition free). **UserProfile** stores **ldap_uid**, **ldap_gid**, and **ldap_groups** (readonly, cached from LDAP sync). The Django Admin "Change User" and "UserProfile" pages show a "Directory (LDAP)" fieldset with this information. On ColdFront startup, **sync_ldap** automatically pushes users/groups to LDAP. The Import Users page supports two modes: New Users (CSV, allocate new UIDs) and Migration (LDIF slapcat dump, preserve existing UIDs/GIDs/passwords).

Minimal LDAP Bootstrap: **install.sh** creates only the bare minimum (root DN, cn=root, munge/slurm system accounts). OUs (**ou=jhu**, **ou=schmidt**, etc.) and user accounts are not hardcoded they come from ColdFront via:

- Option A: Import Users -> Migration (LDIF upload via UI) replays structural entries + users + groups + creates projects/allocations
- Option B: **./start.sh ldap-reinit** (CLI) restores full slapcat dump
- Option C: **sync_ldap** pushes users/groups from ColdFront DB to LDAP

Put the cross-domain admin group (e.g., **PI_XX**) as a DN-based group (for example **groupOfNames** or **groupOfUniqueNames**) and include member DNs from any suffix so membership is unambiguous.

Why this is safe / best-practice

- Disjoint numeric ranges prevent collisions for NSS/PAM and NFS.
- DN-based group membership is unambiguous across suffixes and works well with Keycloak group mappers.

```
Minimal LDIF example:
dn: cn=PI_XX,ou=Groups,dc=arch,dc=cluster
objectClass: top
objectClass: groupOfNames
cn: PI_XX
member: uid=alice,ou=People,ou=jhu,dc=arch,dc=cluster
member: uid=bob,ou=People,dc=schmidt,dc=arch,dc=cluster

Recommendation: place that group in a shared container (e.g., ou=Groups,ou=jhu,dc=arch,dc=cluster) and conf...
```

Additional recommendations:

- Do NOT use **memberUid/posixGroup** for cross-suffix admin groups **memberUid** is just a username and can collide across suffixes.
- Ensure the referenced user entries actually exist (add users before groups, or import groups after users when bootstrapping) so DNs resolve correctly.
- Keycloak: configure the LDAP group mapper to search the shared group container and to honor DN-based membership (works well with **groupOfNames** / **groupOfUniqueNames**).
- Configure Keycloak per-realm to point at the appropriate suffix (or add multiple LDAP providers per realm) so each realm sees the right subtree(s).

start.sh -- Stack Lifecycle Management

Use **start.sh** to manage the full CLOACK stack. It wraps **docker compose** with idempotency checks, host bootstrapping, and service scoping.

Commands

Command	Description
`./start.sh init [dev]`	Create host user/group, prepare dirs, write `.env` (only if missing)
`./start.sh start [dev]`	Pull/build and start all managed services (auto-builds missing images)
`./start.sh restart [dev]`	Stop, rebuild all images (`--no-cache`), then start (preserves data)
`./start.sh stop [dev]`	Stop managed services

`./start.sh destroy [dev]`	Stop and remove managed containers (keeps volumes/data)
`./start.sh rebuild [dev]`	Stop, rebuild all images from scratch (`--no-cache`), then start
`./start.sh populate dev [--partition=<partition>]`	Import departments, setup billing, generate `SlurmJob` (dev only). Flags: `--skip-billing`, `--with-usage`
`./start.sh ldap-reinit [dev]`	If `migration/ldap-*.cat` exists: restore LDAP from backup (passwords patched from `.env`). Otherwise: reinitialize via `install.sh`. Both sync passwords and re-seed ColdFront
`./start.sh cleanup`	Remove unused `arch/*` images not referenced by any container
`./start.sh prune-volumes [dev]`	Stop services and delete OpenLDAP named volumes (data loss -- use with care)
`./start.sh prune-slurm [dev]`	Stop Slurm containers and delete all Slurm volumes (munge key reset)
`./start.sh prune-images`	Remove all Slurm Docker images (`arch/cn-rockylinux`, `arch/slurmd`, etc.)
`./start.sh clean-all`	Full cleanup: stop, destroy, remove arch volumes/images/networks only (safe for co-hosted services). In dev mode also removes home/scratch data. Fresh start
`./start.sh log [SERVICE]`	Tail logs for a service (default: `coldfront`)
`./start.sh reconf`	Push updated `slurm.conf` to all nodes (`scontrol reconfigure`)
`./start.sh test [dev] [MODULE]`	Run Django unit tests (default: all arch_sync tests). Example: `./start.sh test coldfront.plugins.arch_sync.tests.test_billing`
`./start.sh helpdesk {start\ stop\ restart\ status\ destroy\ log\ populate}`	Manage the Helpdesk service. `populate` creates sample queues and tickets (idempotent)
`./start.sh help`	Show usage

Quick start

```
# First-time bootstrap: prepare host dirs and generate .env
./start.sh init dev          # dev
./start.sh init              # prod

# Start the stack
./start.sh start dev         # dev (docker-compose-dev.yml, no /opt/mprov mounts)
./start.sh start             # prod (docker-compose.yml + docker-compose-prod.yml)

# Dry-run: preview what would happen (no changes)
./start.sh start dev --dry-run

# Force a full rebuild + redeploy
FORCE_BUILD=1 FORCE_REDEPLOY=1 ./start.sh start dev

# Restart: stop + rebuild + start (preserves data)
./start.sh restart dev

# Rebuild all images from scratch (--no-cache) and restart
./start.sh rebuild dev

# Reinitialize OpenLDAP and re-sync users into ColdFront
./start.sh ldap-reinit dev   # dev
./start.sh ldap-reinit       # prod

# Default: departments + billing rates/configs + SlurmJob
./start.sh populate dev --partitions premium,scavenger,gpu

# Skip billing rates/AllocationBilling; keep SlurmJob
./start.sh populate dev --partitions premium,scavenger,gpu --skip-billing

# Include SlurmAccountUsagePeriod (billing/usage pages)
./start.sh populate dev --partitions premium,scavenger,gpu --with-usage

# Stop / remove containers (keeps volumes)
./start.sh stop dev
./start.sh destroy dev
```

./start.sh start -- internal flow

Each **start** runs these steps in order:

1. Start **postgres** and **openldap** first (before all other services)
2. Wait for **postgres** to become healthy + **init-databases.sh** to complete
3. Sync DB role passwords (**ALTER USER admin/coldfront/keycloak**) ensures **.env** passwords match postgres, even after **init** regenerated secrets
4. Start all remaining services (**compose_up_scoped**)
5. Post-compose sync re-sync postgres + LDAP passwords (handles container recreation), restart unhealthy app containers if needed
6. Wait for all services to become healthy

Note: LDAP migration from **migration/ldap-*.cat** is not done during **start**. Use **./start.sh ldap-reinit [dev]** after the stack is healthy (see Deploy flows below).

Native docker compose diagnostics

The script intentionally omits status/log/stats commands use **docker compose** directly:

```
docker compose ps                   # service status + health
docker compose logs -f SERVICE      # tail logs for a service
docker compose stats                # live resource usage
```

dev vs production

The optional **dev** argument switches the compose file. All commands accept **[dev]**.

Setting	Production	Dev
Compose files	`docker-compose.yml` + `docker-compose-prod.yml`	`docker-compose-dev.yml`
Slurm host mounts	Yes (`docker-compose-prod.yml`)	No
`PLUGIN_SLURM`	`1`	`0` (default)
`SLURM_NOOP`	`0`	`1` (default)
`SEED_ON_RESTART`	`true` (default)	`true` (default)

Compose file overlay **docker-compose-prod.yml** is automatically appended by **start.sh** when running in production mode. It adds bare-metal Slurm bind mounts to **coldfront** and **qcluster**. Docker Compose merges the files: list fields (**volumes**, **ports**) are appended; scalar fields (**image**, **restart**) are overridden by the overlay. **docker-compose-dev.yml** completely redefines those services so it never inherits the prod mounts.

SEED_ON_RESTART controls whether **seed_initial_data.py** syncs LDAP users into ColdFront on every container start. Set **SEED_ON_RESTART=false** in **.env** to skip on routine restarts. **./start.sh ldap-reinit [dev]** always forces a full sync regardless of this setting.

Environment variables

./start.sh init writes a **.env** file (skipped if already present). Key variables:

```
# Compose file override (alternative to `start dev`)
COMPOSE_FILE_PATH=docker-compose-dev.yml

# Force a full image rebuild
FORCE_BUILD=1

# Force redeployment even if services are already healthy
FORCE_REDEPLOY=1

# Slurm (leave empty on dev; set real paths on production)
SLURM_CONF=/opt/mprov/slurm/current/etc/slurm.conf
SLURM_MPROV=/opt/mprov/slurm/current/bin
PLUGIN_SLURM=1
SLURM_NOOP=0
```

Other auto-detected variables written by **init**:

Variable	Purpose	Auto-detected
`DOCKER_SOCK`	Docker socket path for container management	macOS: `~/.docker/run/docker.sock`, Linux: `/var/run/docker.sock`
`ARCH_HOST_PROJECT_ROOT`	Host project root for bind mounts	`$(pwd)`

JWT token for slurmrestd

start.sh slurm start generates a 30-day JWT token and writes it to **${BASE_ETC}/slurm/slurm_jwt.token**. This file is bind-mounted into coldfront and qcluster containers. The **SlurmClient** reads the token for REST API authentication. A crontab entry (installed by **init**) renews it daily at 3am.

In dev mode, if no **docker-compose-slurm-*.yml** overlay exists, **slurm start** auto-generates a default **docker-compose-slurm-arch-dev.yml** with the standard 4-node cluster (login01, cn-01, cn-02, cn-03) and default partitions (premium, scavenger). In production, the overlay must be deployed from the ColdFront UI.

Note: JWT auth requires Slurm 23.x+. Docker dev currently uses Slurm 22.05 which has a known JWT bug upgrade to 25.11 is planned.

Compose project isolation

Project	Compose file	Services
`arch`	`docker-compose-dev.yml` (or `docker-compose.yml` + `docker-compose.override.yml`)	postgres, openldap, coldfront, keycloak, qcluster, phpldapadmin
`arch-slurm`	`docker-compose-slurm-arch-dev.yml`	slurmctld, slurmdbd, slurmrestd, slurm-db, cn-*, login*

Slurm containers are isolated in their own project so **stop**/**restart** from CLI or UI doesn't affect main services.

Security: password separation

The **.env** file contains several password variables. Two are especially important to keep separate:

Variable	Purpose	Used by
`LDAP_ADMIN_PASS`	Bind DN password (`cn=admin,dc=arch,dc=cluster`) -- full LDAP admin access	ColdFront LDAP bind, Keycloak LDAP federation, `ldapmodify` operations
`JHU_ADMIN_PASSWORD`	User login password (`uid=admin`) -- ColdFront web login	Staff browser login to ColdFront admin

Why they must stay separate: if both use the same password, anyone who knows the ColdFront admin login also has full LDAP admin access (create/delete users, modify ACLs, read all attributes). In production this is a security risk.

Other password variables: **LDAP_ROOT_PASS** (LDAP **cn=root** used internally by **replace_ldif.sh** for LDIF imports), **POSTGRESQL_PASSWORD**, **COLDFRONT_DB_PASSWORD**, **KEYCLOAK_DB_PASSWORD**.

Deploy flows

A. Fresh deploy (no migration)

```
./start.sh clean-all dev
./start.sh init dev
./start.sh start dev
# Slurm cluster -- CLI or UI (Admin -> Clusters -> Add -> Build -> Deploy)
./start.sh slurm create arch-dev
./start.sh slurm start arch-dev
```

B. Deploy + migrate from production

Option 1: CLI

```
./start.sh clean-all dev
./start.sh init dev
./start.sh start dev
./start.sh ldap-reinit dev           # restore LDAP from migration/ldap-*.cat
./start.sh slurm create arch-dev
./start.sh slurm start arch-dev
```

Option 2: UI (after **init + start**)

1. Import Users -> Migration upload slapcat **.cat** file (creates LDAP structure + users + groups + projects)
2. Admin -> Clusters -> Add -> Deploy create and start Slurm cluster
3. Sync Slurm from cluster detail page (creates Slurm accounts)

C. Reinitialize LDAP (no migration backup)

```
# Runs install.sh to create minimal LDAP structure (root + munge + slurm)
./start.sh ldap-reinit dev
```

How **ldap-reinit** works:

- If **migration/ldap-*.cat** exists -> restores from backup via **slapadd**, patches all passwords (**cn=root**, **cn=readonlyroot**, **uid=admin**) with SSHA hashes from **.env**, syncs **olcRootPW** (config DB), seeds ColdFront
- If no cat file -> removes init marker, restarts openldap (re-runs **install.sh**), syncs passwords, seeds ColdFront

D. LDAP cleanup (after migration restore)

Migration dumps may contain test accounts not in **install.sh**. Use **ldap-cleanup.sh** to find and remove them:

```
# Dry-run -- compare migration dump against install.sh, save extras to scripts/ldap-extras.txt
./scripts/ldap-cleanup.sh --dump migration/ldap-2026.04.03.10.cat

# Apply on live LDAP (deletes extra entries)
./scripts/ldap-cleanup.sh --apply
```

OpenLDAP diagnostics (manual):

```
# Verify LDAP entries
source .env
docker exec openldap ldapsearch -H ldapi:/// -x \
  -D "cn=admin,dc=arch,dc=cluster" -w "${LDAP_ADMIN_PASS}" \
  -b "dc=arch,dc=cluster" "(uid=*)" uid | grep "^uid:"

# Force LDAP -> ColdFront re-sync
./start.sh ldap-reinit dev
```

Docker Images

All images use the **arch/** namespace. Build order matters for Slurm images (base first).

Component Versions

Component	Version
ColdFront	1.1.7
Django	4.2
Python	3.10
Slurm	22.05.9
PostgreSQL	16 (supported until Nov 2028; v17 available but no significant benefits for Django/ColdFront)
Keycloak	26.5
MariaDB	10.11
OpenLDAP	1.5.0 (osixia)
Rocky Linux	9 (Slurm base)
Munge	bundled with Slurm

Core Services

Image	Dockerfile	Base	Description
`arch/coldfront`	`coldfront/Dockerfile`	`python:3.10-slim`	Django portal + django-q worker. Runs ColdFront with custom plugins
`arch/openldap`	`openldap/Dockerfile`	`osixia/openldap:1.5.0`	OpenLDAP server with multi-arch support (amd64/arm64)
`arch/keycloak`	`keycloak/Dockerfile`	`keycloak:latest`	Keycloak OIDC with custom themes and LDAP providers
`arch/keycloak-config`	`keycloak/Dockerfile.confi`	`debian:bookworm-slim`	One-shot: configures Keycloak realms and LDAP federation
`arch/postgres`	`postgres/Dockerfile`	`postgres:16`	PostgreSQL with multi-database init (coldfront, keycloak, slurm_jobs)
`arch/phpldapadmin`	`phpldapadmin/Dockerfile`	`osixia/phpldapadmin:latest`	phpLDAPadmin web UI for LDAP browsing

Slurm Services (build order: base -> munge -> slurmdbd -> slurmctld -> slurmd -> slurmrestd)

Image	Dockerfile	Base	Description
`arch/cn-rockylinux`	`slurm/base/Dockerfile`	`rockylinux:9`	Base image: Rocky 9 + Slurm 22.05 + Munge + SSSD + SPANK plugins
`arch/slurm-munge`	`slurm/munge/Dockerfile`	`arch/cn-rockylinux`	One-shot: generates shared munge key in `munge-key` volume
`arch/slurmdbd`	`slurm/slurmdbd/Dockerfile`	`arch/cn-rockylinux`	Slurm accounting daemon (connects to MariaDB)
`arch/slurmctld`	`slurm/slurmctld/Dockerfile`	`arch/cn-rockylinux`	Slurm controller daemon
`arch/slurmd`	`slurm/slurmd/Dockerfile`	`arch/cn-rockylinux`	Slurm worker/login node daemon
`arch/slurmrestd`	`slurm/slurmrestd/Dockerfile`	`arch/cn-rockylinux`	Slurm REST API daemon
`arch/mariadb`	`slurm/mariadb/Dockerfile`	`mariadb:10.11`	MariaDB for Slurm accounting (port 3307)

Other

Image	Dockerfile	Base	Description
`arch/ondemand`	`ondemand/Dockerfile`	`rockylinux:9`	Open OnDemand web portal

Billing & Invoicing System

ARCH includes a comprehensive billing system built into ColdFront to track resource usage (compute, GPU, storage, backup) and manage invoicing for JHU and Schmidt Sciences allocations.

Key Features

Two-Track Billing Model

The system supports three billing types:

Type	For	How It Works
Pay Per Use	PIs without hardware credit pool	Full rate charged per hour (compute + O&M bundled)
Credit Holder	PIs who purchased hardware	Credits cover compute hours only; O&M (power/cooling) charged separately in cash
No Charge	Default, scavenger, and Schmidt allocations	Usage tracked but not billed ($0 total)

Important: Credits do NOT pay O&M bills. The system UI clearly separates credit consumption from cash charges to prevent confusion.

Billing eligibility is controlled by **AllocationBilling.is_billable** (set automatically by signal):

- **is_billable=True** -> allocation is charged (Pay Per Use or Credit Holder)
- **is_billable=False** -> allocation is tracked with **billing_type='no_charge'** and **payment_status='no_charge'**

Allocations with **is_billable=False** still appear in billing reports showing real usage (hours, rate) but with $0 total and "No Charge" label. Staff can toggle **is_billable** via Django Admin. The billing report includes a "Billable only" checkbox (default ON) to filter the view.

Staff Billing Report

Superusers and billing staff can:

- View filterable billing report with all usage and charges
- Filter by: PI, project, user, school, department, affiliation, billing type, date range
- See summary metrics: total jobs, CPU hours, GPU hours, average hours/job, total cost
- Download CSV export for accounting integration
- Access at: **/billing/report/**

PI Usage & Billing Dashboard

PIs can:

- See credit balance and estimated runway (days until exhaustion)
- View separate O&M charges in current billing period (30-day rolling window)
- Track recent usage by period
- Download printable invoices with IOS/cost center number
- Access at: **/billing/my-usage/**

Rate Management

Superusers can:

- Define rates for compute (CPU/GPU), storage, and backup services
- Set different rates per cluster and partition
- Create rate history with effective date ranges

- Rates are stored separately for:
- cpu_full / gpu_full full rates for pay-per-use allocations
- cpu_om / gpu_om O&M only rates for credit holders

Hardware Credit Pools

Superusers track hardware purchases:

- Record hardware value in dollars
- Convert to compute credit hours (e.g., "4x H200 GPU" -> "87,600 GPU-hours")
- Track credit consumption and remaining balance
- Set expiry dates if credits sunset

IOS Numbers & Department Tracking

Allocations can have:

- IOS Number cost center / account number for JHU service center
- Department organizational affiliation (e.g., "Computer Science", "Biology")
- Billing Type pay-per-use, credit holder, or no charge
- Provisioned Storage/Backup manual entry for non-compute charges

Usage Guide

Setting Up Billing

1. Enable billing collection (environment variable):

```
export BILLING_COLLECT_ENABLED=1
```

2. Create billing rates (Django Admin or CLI):

```
# Via Admin: /admin/arch_sync/billingrate/add/
   # Create rates for cpu_full, gpu_full, cpu_om, gpu_om, storage, backup
```

3. Import departments (optional, for organizational grouping):

```
python manage.py import_departments departments.csv
   # CSV format: name,code,school,affiliation
```

4. Assign PIs to departments (optional):

```
python manage.py assign_pi_departments pi_departments.csv
   # CSV format: username,department_name
```

5. Set allocations as billable:

- Open Allocation Detail
- Scroll to "Billing Info" card

- Set IOS number and billing type (staff/superuser only edit form is not shown to PIs or managers)
- Save

```
Automatic creation on approval: When an allocation transitions to Active (approved by admin), an `Allocatio...
```

```
Visibility rules for Billing Info card (`allocation_detail.html`):

The card is shown when any of these conditions is true:
- `request.user.is_superuser` or `can_approve_deny` -- superusers and billing staff
- `request.user == allocation.project.pi` -- the PI who owns the project
- `is_allowed_to_update_project` -- project managers (`ProjectPermission.UPDATE`)

| Role | Can see IO number & billing type? | Can edit (IO / billing type)? |
|------|-----------------------------------|-------------------------------|
| Superuser / Staff (`is_superuser` or `can_approve_deny`) | Yes | Yes |
| PI (`request.user == allocation.project.pi`) | Yes (read-only) | No |
| Project Manager (`ProjectPermission.UPDATE`) | Yes (read-only) | No |
| Regular allocation user | No | No |

The edit form (IO Number input + Update Billing button) is wrapped in its own
`{% if request.user.is_superuser or can_approve_deny %}` guard and is never
shown to PIs or managers.
```

Collecting Billing Data

Billing usage data is stored in two models:

Model	Source	Purpose
`SlurmJob`	`collect_sacct` (from `sacct`)	Per-job records -- source of truth for Job Metrics dashboard
`SlurmAccountUsagePeriod`	`slurm2coldfront()` (aggregates from `SlurmJob`)	Per-period usage snapshots -- source of truth for billing

Job data collection (**collect_sacct**):

```
# Collect last 7 days of per-job data from sacct
python manage.py collect_sacct --current-week

# Collect current month
python manage.py collect_sacct --current-month

# Import from a pipe-delimited sacct dump file (for dev/testing)
python manage.py collect_sacct --from-file coldfront/templates/jobs_usage.csv

# Dry-run (parse and log, no DB writes)
python manage.py collect_sacct --dry-run
```

Billing aggregation (**slurm2coldfront**):

```
# Aggregates SlurmJob data into SlurmAccountUsagePeriod per allocation
# Called automatically by run_slurm_sync (--direction both)
python manage.py sync_slurm --direction both
```

Automatic scheduling: A cron job in the Slurm entrypoint runs **sync_slurm direction both** daily at 3:30 AM (configurable via **SLURM_SYNC_CRON_EXPR**). This calls **collect_sacct current-week** at the end, so job data stays fresh.

Legacy billing collection (still available):

```
python manage.py collect_billing --current-period
python manage.py collect_billing --current-period --dry-run
python manage.py collect_billing --current-period --pi rdesouz4
```

Printable Invoices

PIs can view and print invoices from their dashboard:

- Access: **/billing/my-invoice//**
- Browser print (Ctrl+P or Cmd+P) -> Save as PDF
- Invoice shows two sections:

1. Compute Credits Used (hardware-backed hours, NOT a cash charge)
2. O&M Charges Due (real dollars owed for power/cooling)

- Includes IOS number for payment routing

Example Data & Quick Test

We include example data to test the billing system (see **BILLING_SETUP_GUIDE.md** for full details):

Departments included (12 total):

- Whiting School of Engineering: CS, ECE, Applied Physics, ME, BME
- Krieger School of Arts & Sciences: Biology, Chemistry, Physics
- School of Medicine: Neuroscience, MBG
- Schmidt Science: Structural Biology, Computational Biology

PIs included (10 total):

- rdesouz4 (Computer Science)
- jsmith5 (Biology), mchen2 (Applied Physics), agarcia7 (ECE)
- bpatel9 (Chemistry), kwilson3 (Neuroscience)
- dgonzalez1 (Structural Biology), hjones8 (Computational Biology)
- lrodriguez4 (Physics), ekim6 (Mechanical Engineering)

Example rates created (6 total):

- CPU Full Rate: $0.42/hr | CPU O&M Only: $0.08/hr
- GPU Full Rate: $3.50/hr | GPU O&M Only: $0.65/hr
- Storage: $0.012/TB/month | Backup: $0.006/TB/month

To populate example data:

```
# Import departments (12)
python manage.py import_departments examples_departments.csv

# Assign PIs to departments (10)
python manage.py assign_pi_departments examples_pi_departments.csv

# Create billing rates (6)
python manage.py shell < populate_example_rates.py

# Then in ColdFront UI:
# 1. Go to each PI's allocation
# 2. Scroll to "Billing Info" card
# 3. Set IOS Number (e.g., 1010500000) and Billing Type
# 4. Click Update Billing

# Collect usage
python manage.py collect_billing --current-period

# View reports
# Staff: /billing/report/
# PI: /billing/my-usage/
```

See **BILLING_SETUP_GUIDE.md** for detailed instructions and troubleshooting.

Staff Workflows

View all usage:

```
Staff menu -> Billing Report
```

Manage rates:

```
Staff menu -> Rate Management -> Add Rate
```

Track hardware credits:

```
Staff menu -> Hardware Credits -> Add Credit Pool
```

Set allocation billing info:

```
Allocation Detail -> Billing Info card -> Set Billing
```

Job Usage Metrics Dashboard

An XDMoD-style analytics dashboard for job usage, available at **/billing/jobs/**:

- Group by: User, Account, Partition, QOS, State, Department (staff only), School (staff only)
- Sortable columns: CPU Hours, GPU Hours, Wall Hours, CPU Hrs/Job, Avg Wait, Jobs, Users, Avg CPUs, % of CPU Total
- Summary cards: Total CPU/Wall hours, job count, active users, avg wait/wall per job; GPU Hours card only appears when **total_gpu > 0**
- CSV export and Raw Jobs detail view with pagination
- Access control data is scoped by role:

Role	Jobs visible
Staff / Superuser	All jobs across all accounts
PI (`userprofile.is_pi`)	Their own jobs + all jobs under accounts in their allocations
Regular user	Only their own jobs (`user=username`)

Data Model

Model	Purpose
`BillingRate`	Rate catalog with effective dates (CPU/GPU/storage/backup)
`HardwareCreditPool`	Hardware purchases converted to credit hours
`AllocationBilling`	Billing config per allocation (IOS, type, storage, backup)
`SlurmJob`	Per-job record from sacct -- source of truth for Job Metrics dashboard
`SlurmAccountUsagePeriod`	Per-period usage snapshot (CPU, GPU, job counts) -- billing source of truth
`Department`	Organizational hierarchy for grouping PIs

Extended models:

- **UserProfile** now has **department** FK for filtering by school/department

Configuration

Relevant environment variables:

```
# Enable billing data collection from Slurm
BILLING_COLLECT_ENABLED=1

# Invoice feature (upstream ColdFront)
INVOICE_ENABLED=1
INVOICE_DEFAULT_STATUS=Payment Pending

# Slurm sync (required for billing data)
SLURM_BIN=/opt/mprov/slurm/current/bin
SLURM_NOOP=0
```

URL Endpoints

Endpoint	Access	Purpose
`/billing/report/`	Staff/Superuser	Filterable usage report
`/billing/report/csv/`	Staff/Superuser	CSV export
`/billing/rates/`	Staff/Superuser	Rate catalog
`/billing/rates/create/`	Superuser	Add rate
`/billing/rates/<id>/edit/`	Superuser	Edit rate
`/billing/credits/`	Staff/Superuser	Hardware credit pools
`/billing/credits/create/`	Staff/Superuser	Add credit pool
`/billing/my-usage/`	PI	Usage dashboard
`/billing/my-invoice/<allocation-id>/`	PI	Printable invoice
`/billing/jobs/`	PI/Staff	Job Usage Metrics dashboard (group-by analytics)
`/billing/jobs/detail/`	PI/Staff	Raw job list with filters and pagination
`/billing/jobs/csv/`	PI/Staff	CSV export of job metrics

`/allocation/<id>/`	All users	Allocation Detail -- Billing Info card visible to superuser, staff, PI, and project managers (read-only for PI/manager; edit form for staff/superuser only)

Examples

Example 1: Credit Holder Invoice

PI Ricardo purchased 4x H200 GPUs -> granted 87,600 GPU-hours of credit.

In a billing period:

- GPU usage: 5,000 hours
- O&M rate: $0.30/hour
- O&M charge: $1,500 (real dollars owed)
- Credits consumed: 5,000 hours
- Invoice shows:
- Section 1 (Credits): 5,000 GPU-hours consumed (no charge, from credit pool)
- Section 2 (O&M): $1,500 due in cash

Example 2: Pay-Per-Use Allocation

PI Jsmith does not have a hardware purchase, uses pay-per-use rate.

In a billing period:

- CPU usage: 10,000 hours
- Full CPU rate: $0.05/hour
- Total charge: $500 (compute + O&M bundled)
- Invoice shows:
- Single line: 10,000 CPU-hours @ $0.05/hour = $500

Example 3: Filter by Department

Staff runs billing report filtered by department "Computer Science":

- See all allocations in that department
- View usage and costs grouped by PI and project
- Export to CSV for departmental billing

Migration & First-Time Setup

1. Apply database migrations:

```
python manage.py migrate
```

2. Create initial rates via Django Admin:

- Log in as superuser
- Go to **/admin/arch_sync/billingrate/add/**

- Add rates for each service type and effective period

3. Mark allocations as billable:

- Open each allocation in Allocation Detail
- Click "Set Billing" in the Billing Info card
- Enter IOS number and select billing type

4. Collect baseline data:

```
python manage.py collect_billing --current-period
```

5. Monitor via:

- Staff -> Billing Report (usage overview)
- Staff -> Hardware Credits (credit consumption)
- PI -> My Usage & Billing (personal dashboard)

Troubleshooting

No usage data appearing?

- Verify **BILLING_COLLECT_ENABLED=1** is set
- Check Slurm **sreport** binary is accessible: **which sreport**
- Run **collect_billing** **dry-run** to debug Slurm queries
- Check allocation is marked "Active" status

Allocation not in billing report?

- Verify **AllocationBilling.is_billable=True** is set (via Allocation Detail -> Billing Info)
- Ensure allocation has usage data (run **collect_billing**)

Wrong rates showing?

- Verify rate **effective_from** date is on or before the billing period
- Check rate **effective_to** date is after the period (or NULL for ongoing rates)
- Confirm cluster/partition match your allocation

Populating Synthetic Data for Testing

For development and testing, use **./start.sh populate dev** to seed departments, billing structure, and SLURM usage data in one command.

Models

Model	What	Page
`Affiliation`	Organizational domain (JHU, Schmidt) -- managed via Admin, FK on UserProfile/Department/School. Includes `next_uid`/`next_gid` counters for centralized UID/GID allocation	Admin
`BillingRate`	Rate catalog: cost per CPU/GPU/storage/backup per partition (e.g. `$0.35/hr CPU`, `$1.20/hr GPU`)	`/billing/rates/`
`AllocationBilling`	Per-allocation config: IO number, billing type, is_billable flag, storage/backup TB, department	`/allocation/<id>/`

`SlurmAccountUsagePeriod`	Monthly aggregated usage: CPU hours, GPU hours, job count per account + payment status	`/billing/report/` `/billing/my-usage/`
`SlurmJob`	Individual job: user, partition, CPU/GPU used, duration, state	`/jobs/metrics/`

Quick Start

```
# Default: departments + BillingRate/AllocationBilling + SlurmJob
./start.sh populate dev --partitions premium,scavenger,gpu

# Só jobs, sem billing
./start.sh populate dev --partitions premium,scavenger,gpu --skip-billing

# Tudo incluindo SlurmAccountUsagePeriod
./start.sh populate dev --partitions premium,scavenger,gpu --with-usage

# Só departamentos + SlurmAccountUsagePeriod + SlurmJob (sem billing rates/configs)
./start.sh populate dev --partitions premium,scavenger,gpu --skip-billing --with-usage
```

What populate does (in order)

Step	Command	Creates
1	Copy `examples_*.csv`	`departments.csv`, `pi_departments.csv` (skip if already present)
2	`import_departments`	`Department` records from `departments.csv`
3	`populate_fake_billing_data --no-usage`	`BillingRate`, `AllocationBilling`, `HardwareCreditPool` (skipped with `--skip-billing`)
4	`generate_sacct_dump.py` + `collect_sacct --clear`	`SlurmJob` records (always runs)
5	`generate_slurm_usage_data`	`SlurmAccountUsagePeriod` (only with `--with-usage`)

Flags

Flag	`BillingRate`/`AllocationBilling`	`SlurmAccountUsagePeriod`	`SlurmJob`
(default)	v	x	v
`--skip-billing`	x	x	v
`--with-usage`	v	v	v
`--skip-billing --with-usage`	x	v	v

Individual commands (via docker exec)

Import departments only:

```
# From default CSV
docker exec -it coldfront python -m django import_departments \
  /opt/arch/coldfront/templates/departments.csv

# From custom CSV (copy file first)
docker cp /path/to/my_departments.csv coldfront:/tmp/departments.csv
docker exec -it coldfront python -m django import_departments /tmp/departments.csv --dry-run
docker exec -it coldfront python -m django import_departments /tmp/departments.csv
```

Regenerate SlurmJob records only:

```
docker exec -it -e SLURM_PARTITIONS="premium,scavenger,gpu" coldfront bash -c "
  python -m django shell < /opt/arch/coldfront/templates/generate_sacct_dump.py && \
  python -m django collect_sacct --clear --from-file /opt/arch/coldfront/templates/sacct_dump.txt
"
```

Clear all SlurmJob records:

```
docker exec -it coldfront python -m django collect_sacct --clear --from-file /dev/null
```

Assign departments to PIs:

```
# CSV format: username,department_names (semicolon-separated)
docker exec -it coldfront python -m django assign_pi_departments \
  /opt/arch/coldfront/templates/pi_departments.csv
```

Expected Results

After populating, you should see:

In Billing Report (**/billing/report/**):

- 4,200+ rows of usage data
- 700+ allocations
- Spanning 6 months of history
- Total cost across all allocations: $100,000+
- Filterable by: PI, department, school, affiliation, billing type

In PI Dashboard (**/billing/my-usage/**):

- Credit balance for hardware credit holders
- O&M charges owed for all allocations
- Usage trends over 6 months
- Estimated runway (days until credits expire)

In Allocations:

- Each allocation shows IOS number and billing type
- Department assignment visible
- Billing history available

CSV File Formats

slurm_usage.csv

```
allocation_id,account,start_date,end_date,cpu_hours,gpu_hours,job_count,partition_name
1000,alloc_1000,2025-12-01,2025-12-31,45000.5,12000.75,1560,gpu
1001,alloc_1001,2025-12-01,2025-12-31,21000.5,0.0,450,cpu
```

pi_departments.csv

```
username,department_names
jbiondo2,Computer Science;Applied Physics
rdesouz4,Computer Science;Biomedical Engineering
ssci-huan,Data Science;Computational Biology
```

departments.csv (for import_departments command)

```
name,code,school,school_code,affiliation
"Computer Science","CS","Whiting School of Engineering","wse","jhu"
"Data Science","DS","Schmidt Sciences Institute","schmidt","schmidt"
```

Billing Report CSV Files (Generated by generate_billing_report command)

The billing system generates three complementary CSV reports saved to **/opt/arch/coldfront/report/**:

1. billing_YYYY-MM_detail.csv Full breakdown per allocation

```
PI,Project,Allocation ID,Department,IO Number,Month,Jobs,CPU Hours,GPU Hours,CPU Cost,GPU Cost,Storage (TB)...
arochab1,arochab1,1299,Computer Science,NO_IO,2026-03,10,3532.70,0.00,176.63,0.00,0.00,0.00,0.00,0.00,176.63
bblanch6,bblanch6,1300,Computer Science,NO_IO,2026-03,28,5404.41,0.00,270.22,0.00,0.00,0.00,0.00,0.00,270.22
```

Use case: Detailed allocation-level tracking for finance/compliance audits.

2. billing_YYYY-MM_summary.csv Simplified format (PI, IO, Total ONLY)

```
PI,IO,Total
arochab1,NO_IO,$176.63
bblanch6,NO_IO,$270.22
jperafa1,NO_IO,$874.96
ssci-adityag,NO_IO,$1740.32
```

Use case: Quick summary for executive reports, dashboard displays, or simplified invoicing.

3. billing_YYYY-MM_by_io.csv Aggregated by IO number with all metrics

```
PI,IO Number,Jobs,CPU Hours,GPU Hours,Node Hours,CPU Cost,GPU Cost,Storage (TB),Storage Cost,Backup (TB),Ba...
arochab1,NO_IO,10,3532.70,0.00,180.60,$176.63,$0.00,0.00,$0.00,0.00,$0.00,$176.63
bblanch6,NO_IO,28,5404.41,0.00,289.80,$270.22,$0.00,0.00,$0.00,0.00,$0.00,$270.22
```

Use case: IO-based billing analysis, cost allocation by infrastructure component.

Generation Commands:

```
# Generate all three CSV reports for a specific month
python manage.py generate_billing_report --month 2026-03 --exclude-admin-pi

# With custom rates
python manage.py generate_billing_report --month 2026-03 \
  --cpu-rate 0.05 \
  --gpu-rate 0.50 \
  --storage-rate 10.0 \
  --backup-rate 5.0

# Generate text invoices (one TXT file per allocation)
python manage.py generate_text_invoices --month 2026-03

# Generate text invoices AND convert to PDF
python manage.py generate_text_invoices --month 2026-03 --to-pdf
```

Customization

To adjust data generation, edit **coldfront/templates/generate_large_usage.py**:

```
# Change allocation type distribution
allocation_types = [
    {'name': 'gpu_intensive', 'cpu': (40000, 100000), 'gpu': (20000, 60000), ...},
    {'name': 'cpu_only', 'cpu': (20000, 50000), 'gpu': (0, 0), ...},
    # Add more types...
]

# Change date ranges
for month_offset in range(6):  # 6 months -> change to 12 for 1 year
```

Then regenerate:

```
python3 coldfront/templates/generate_large_usage.py
```

Troubleshooting

"No usage data appearing in report?"

- Verify allocations are marked as billable (Allocation Detail -> Billing Info -> check box)
- Run: **docker compose -f docker-compose-dev.yml exec coldfront python manage.py shell**

```
from coldfront.plugins.arch_sync.models import SlurmAccountUsagePeriod
  SlurmAccountUsagePeriod.objects.count()  # should be > 1000
```

"Import says 'allocation not found'"

- Verify allocations were created by populate_fake_billing_data
- Check allocation IDs in CSV match database: **SELECT id FROM allocation LIMIT 5;**

"Charts/totals look wrong?"

- Verify rates exist and have correct effective dates
- Check that allocations have **billing*is_billable=True***

10. Slurm CLI Utilities

These utilities are installed in the base Slurm image (**arch/cn-rockylinux**) at **/usr/local/bin/** and available on all nodes. All ARCH tools support **-M** for multi-cluster queries.

stree -- Account Hierarchy

```
# Show your own account tree (auto-detects $USER)
stree

# Show your tree with user associations
stree --full

# Show subtree (admin or own accounts only)
stree -r jhu

# Full hierarchy from root (admin only)
stree -r root --full
```

```
Note: Regular users can only view accounts they belong to. `stree -r root` requires admin privileges (root/...
```

Run **stree -r root** on a login node to see the live hierarchy. Example output:

```
root
|-- jhu   [Fairshare=20]
|   |-- wse   (school)
|   |   +-- cs   (department)
|   |       |-- rdesouz4   [100000]  <- PI default (QOS: scavenger)
|   |       |   +-- rdesouz4
|   |       |   +-- rdesouz4_proj1   [100000]  <- custom project (QOS: normal,premium)
|   |       |       |-- rdesouz4_proj1_A   [100000]  <- subproject
|   |       |       |   +-- rdesouz4, arochab1
|   |       |       +-- rdesouz4_proj1_B   [100000]
|   |       |           +-- rdesouz4
|   |       +-- arochab1   [100000]
|   |           +-- arochab1
|   +-- it   (school)
|       +-- arch   (department)
|           +-- bblanch6   [100000]
|               +-- bblanch6
+-- schmidt   [Fairshare=80]
    +-- ss   (Schmidt Sciences)
        |-- ssci-jdoe   [100000]  <- always free (ssci-* prefix)
        |   +-- ssci-jdoe
        +-- ssci-gomes   [100000]
            +-- ssci-gomes

[N] = Core Usage (Hours) / Fairshare weight
Use stree --full to include users
```

Troubleshooting:

- PIs flat under root (not under school/dept)? -> PI project has no Department set. Fix: assign a department in ColdFront Project Detail, then run **sync_slurm**
- Hierarchy not updating? -> Run: **docker exec coldfront coldfront sync_slurm**
- Need to reparent an account manually? -> **sacctmgr modify account where name=ACCT set parent=NEWPARENT -i** (requires **where** keyword)

interact -- Interactive Sessions

```
# List available partitions + your accounts & QOS
interact -l

# Start a session (1 core, 1 hour, scavenger)
interact -p scavenger -n 1

# 4 cores, 16GB, premium with specific account
interact -p premium -n 4 -m 16G -a mypi_proj1

# GPU session
interact -p gpu -g 1 -n 4 -m 32G -t 2:00:00
```

Built-in checks: validates walltime against partition limit, auto-detects GPU partitions, shows weekly cap budget remaining before submitting.

sbalance -- Usage Report

```
# Show usage for all your accounts
sbalance

# Show weekly spend cap status (cap vs usage in $)
sbalance --week

# Show usage for a specific PI or user
sbalance -p rdesouz4
sbalance -u rdesouz4

# Hide accounts with zero usage
sbalance --hide-zero
```

Troubleshooting:

- All zeros? -> No jobs have run yet, or **collect_sacct** hasn't imported them
- Missing accounts? -> Run **sync_slurm** to ensure ColdFront allocations are synced to Slurm

sqos -- QOS Definitions

```
# Show all QOS definitions
sqos all

# Show a specific QOS
sqos name=normal

# Show QOS for a specific account or user
sqos account=rdesouz4
sqos rdesouz4
```

```
Old name `slurmqos` still works as a symlink.
```

Other Tools

Command	Purpose

`sqme`	Show your active jobs (formatted `squeue`)
`seff <jobid>`	Show CPU and memory efficiency for a completed job
`sfeatures`	Show node features, GRES, and partition info
`sbrief`	Job state summary (count by Running, Pending, etc.)
`sassoc`	Show your associations (account, QOS, fairshare)
`sassoc --clusters`	List all registered Slurm clusters
`sadmin priority <jobid>`	Set high priority (admin only)
`sadmin addtime <jobid> <time>`	Add time to running job (admin only)

sacctmgr -- Common Debugging Commands

```
# List all associations for an account
sacctmgr -n -P list assoc where account=ACCOUNT format=Cluster,Account,User,Partition,QOS

# List all accounts with parents
sacctmgr -n -P list associations where user= format=Account,ParentName,Share

# List accounts under a specific parent
sacctmgr -n list account where parent=schmidt format=Account,Description

# Show account details
sacctmgr show account name=rdesouz4 withassoc format=Account,User,QOS,GrpTRESMins

# Reparent an account (requires 'where' keyword!)
sacctmgr modify account where name=rdesouz4 set parent=nodep -i
```

```
Note: `sacctmgr list account format=ParentName` returns empty. Use `list associations where user=` to get p...
```

start.sh slurm -- Cluster Management

Option A: CLI

```
./start.sh slurm create             # Generate compose overlay + DB record
./start.sh slurm start              # Build images (if needed) + compose up
./start.sh slurm status             # Node state + job queue
./start.sh slurm stop               # Stop Slurm services
./start.sh slurm restart            # Restart (slurmdbd -> slurmctld -> nodes)
./start.sh slurm destroy            # Stop + remove containers, volumes, overlay
```

Option B: UI (Admin -> Clusters)

1. Add Cluster register cluster with name, type, and deployment model preset
2. Build Images compile Slurm base + service images
3. Deploy generate slurm.conf + compose overlay + start containers
4. Stop / Restart manage running cluster from detail page

Design: Docker exec vs local binaries

All Slurm CLI commands (**sacctmgr**, **sinfo**, **sreport**, etc.) run via **docker exec {cluster}-slurmctld** rather than local binaries:

	Docker exec	Local binaries + volumes
Simplicity	1 function, no extra volumes	3 init containers + 2 volumes
Multi-cluster	Routes to correct slurmctld	Local binary can't resolve cluster hostname
Dependencies	Zero -- only Docker SDK	Needs munge key, slurm libs, ldconfig
Maintenance	None	Slurm upgrade requires rebuild of init container

Bare-metal (prod) uses local binaries directly handled by the **else** branch in **slurm_command()**.

Troubleshooting:

- Nodes showing "down" after restart? -> Wait 10 seconds (auto-resume runs in background). If still down: **scontrol update nodename=cn-01 state=resume**
- "No docker-compose-slurm-*.yml found"? -> Deploy the cluster from the ColdFront UI first (Clusters -> Build Images -> Deploy)
- Init containers showing "Exited (0)" in red? -> Normal. **munge-init** is a one-shot container that generates the shared munge key. It consumes zero CPU/memory after exit.

Testing

Commands

```
# Run all tests (108 tests)
./start.sh test

# Full stack integration test (16 phases: containers, LDAP, Slurm, jobs, billing, QOS, sync)
./start.sh test deploy

# List available test modules with counts
./start.sh test list

# Show test help with examples
./start.sh test --help

# Run a specific test module
./start.sh test coldfront.plugins.arch_sync.tests.test_billing

# Run a specific test class
./start.sh test coldfront.plugins.arch_sync.tests.test_models.CoreHoursResetSettingsTest

# Run a specific test method
./start.sh test coldfront.plugins.arch_sync.tests.test_signals.DepartmentSignalTest.test_billing_dept_propa...
```

Available Test Modules

Module	Type	What It Tests
`arch_sync.tests.test_billing`	SimpleTestCase	`dollars_to_grptres_mins`, `grptres_mins_to_dollars`, `hours_to_dollars`, `SYSTEM_USERNAMES`
`arch_sync.tests.test_models`	TestCase	`CoreHoursResetSettings.compute_next_reset()` (6 periods), `PaymentStatus` ordering
`arch_sync.tests.test_slurm_sync`	TestCase	`SlurmNode/SlurmPartition.slurm_conf_line` (AllowQos, TRESBillingWeights), weekly cap conversion
`arch_sync.tests.test_signals`	TestCase	Department propagation signals, AllocationBilling auto-creation
`arch_sync.tests.test_ldap`	SimpleTestCase	DN parsing, SSHA hash, tree config, UID/GID allocation, email comparison
`core.user.tests`	TestCase	User registration, PI request flow

Full module path prefix: **coldfront.plugins.** (for arch_sync) or **coldfront.** (for core).

Example Output

```
$ ./start.sh test
[+] Running Django tests: coldfront.plugins.arch_sync.tests

  v  Applying contenttypes.0001_initial
  v  Applying auth.0001_initial
  ...
  v  test_decimal (coldfront.plugins.arch_sync.tests.test_billing.DollarsToGrpTRESminsTest)
  v  test_one_dollar (coldfront.plugins.arch_sync.tests.test_billing.DollarsToGrpTRESminsTest)
  v  test_dept_propagates_to_default_project (...) -- Setting billing dept propagates to Project.department.
  v  test_simple_dn (coldfront.plugins.arch_sync.tests.test_ldap.UidFromDnTest)
  ...

----------------------------------------------------------------------
Ran 108 tests in 0.729s

OK
```

Note: **SimpleTestCase** tests (test_billing, test_ldap) run without a database. **TestCase** tests require CREATEDB permission (granted automatically by **./start.sh test**).

User Guide

Regular Users

Users join existing projects and use allocations managed by PIs.

Action	How
View your projects	Projects (sidebar)
View your allocations	Click a project, see Allocations section
Join a project	PI adds you via Add Users on the project page
Check Slurm account	Allocation detail shows `slurm_account_name`
Submit a job (web)	Jobs > Submit -- choose partition + account
View job history	Jobs > Metrics -- filter by your username
Request more hours	Allocation detail > Request Change

Principal Investigators (PIs)

PIs own projects, manage members, and control allocations.

Action	How
Create a project	Projects > Create Project -- set title, abbreviation, department, IO Number
Request storage/backup	Check Storage/Backup boxes on project create/update form
Add users	Project detail > Add Users -- search by username, select allocations
Set weekly spend cap	Allocation detail > Set Weekly Cap ($)
View billing	My Usage & Billing (sidebar)
Request allocation change	Allocation detail > Request Change (e.g. more Core Hours)

Staff Tools

Tool	Purpose
Manage Users	Toggle PI status, assign cluster/affiliation scope, impersonate users
Assign Staff Scope	Set cluster scope (manage clusters) and affiliation scope (manage users/PIs)
Staff User Profile	Edit name/email, set LDAP password, toggle shell access (`/bin/bash` vs `/bin/false`)
Promote to PI	Creates default project + allocation + Slurm account automatically
Elevated approvals	Grant via Admin > User profiles > "Elevate to superuser for approvals" -- allows scoped staff to approve/deny without full superuser
Staff Help	In-app reference at /staff/help/ with downloadable PDF
Service Health	Real-time status of all services at /user/service-health/

Delegated Staff Administration

Staff members can be given two independent scopes via the "Assign Staff Scope" page:

Scope	What It Controls
Cluster Scope	View cluster status, drain/resume nodes for assigned clusters
Affiliation Scope	See/manage users, PIs, projects, allocations of assigned affiliations

Multiple clusters and affiliations can be assigned (checkboxes). Superusers always see everything. Scoped staff see a simplified Manage Users page with only their affiliation's users. Deploy/stop/restart operations remain superuser-only.

Best Practices

- Keep projects tidy archive inactive projects
- Use descriptive abbreviations they become Slurm account names
- Plan ahead for renewals submit extension requests before allocations expire
- Set weekly spend caps on paid projects to prevent overspending

---

Multi-Cluster Architecture

CLOACK supports two deployment models for scaling beyond a single cluster. Both use the same ColdFront codebase the **SlurmCluster** model and **-M** flag routing are already implemented.

Docker Multi-Cluster: Each cluster gets its own prefixed containers (**{name}-slurmctld**, **{name}-cn-01**) and volumes (**{name}_slurm_state**). Shared services (**slurmdbd**, **slurmrestd**, **slurm-db**) run as single instances supporting federation. Multiple clusters can run simultaneously.

Model A -- Single Cluster (Recommended for Start)

One slurmctld manages all nodes. Different hardware is separated by partitions.

```
+-------------+
                    |  slurmdbd   |
                    |  slurmrestd |
                    L------+------+
                           |
                    +------+------+
                    |  slurmctld  |
                    |   "arch"    |
                    L------+------+
              +------------+------------+
              ?            ?            ?
        +----------+ +----------+ +----------+
        | cpu-01.. | | gpu-01.. | |  hm-01.. |
        | (600 CPU)| |(200 GPU) | | (150 HM) |
        L----------+ L----------+ L----------+

Partitions: scavenger, premium, gpu-a100, high-memory, dev
```

Benefit	Detail
Simple operations	One slurmctld to manage, monitor, and upgrade
Unified scheduling	Scheduler sees all resources, optimal placement
Easy configuration	One slurm.conf, one set of partitions
Scales to ~10K nodes	Single slurmctld handles 1000 nodes easily

Drawback	Detail
Single point of failure	slurmctld crash affects all users
Maintenance downtime	Upgrade/restart impacts all jobs
Mixed policies	All hardware shares the same scheduling policies

Model B -- Federation (When You Need Isolation)

Multiple slurmctld instances sharing one slurmdbd. Each cluster has its own partitions and scheduling policies. Users submit cross-cluster with **sbatch -M cluster_name**.

```
+-------------+
                    |  slurmdbd   | <- shared accounting DB
                    |  slurmrestd | <- single REST API for all clusters
                    L------+------+
              +------------+------------+
              ?            ?            ?
        +----------+ +----------+ +----------+
        |slurmctld | |slurmctld | |slurmctld |
        |"arch-cpu"| |"arch-gpu"| |"arch-dev"|
        L----+-----+ L----+-----+ L----+-----+
             ?            ?            ?
        +--------+  +----------+  +--------+
        |600 CPU |  | 200 GPU  |  | 50 dev |
        | nodes  |  |  nodes   |  | nodes  |
        L--------+  L----------+  L--------+
              ?            ?            ?
              L------------+------------+
                     +-----+-----+
                     |  login01  | <- shared login node
                     L-----------+
```

Benefit	Detail
Fault isolation	arch-gpu crash does not affect arch-cpu jobs
Independent maintenance	Upgrade GPU cluster without touching CPU cluster
Optimized scheduling	Each cluster tuned for its hardware (GPU policies differ from CPU)

Dev isolation	Users test on dev cluster without impacting production
Scalable beyond 10K nodes	Each cluster handles up to ~10K nodes independently
Unified billing	Shared slurmdbd = single accounting database across all clusters
Cross-cluster submission	`sbatch -M arch-gpu` from any login node

Drawback	Detail
Operational complexity	Multiple slurmctld instances to manage
Configuration overhead	Separate slurm.conf per cluster
Resource fragmentation	Scheduler in arch-cpu cannot use idle arch-gpu nodes

Comparison

Aspect	Single Cluster	Federation
Scheduling	1 scheduler for all jobs	Dedicated scheduler per cluster
Scalability	~10K nodes per slurmctld	~10K nodes **per cluster**
Failure blast radius	All users affected	Only users of that cluster
Maintenance	Downtime affects everyone	Upgrade one cluster at a time
Partitions	Shared namespace	Isolated per cluster (no name conflicts)
Hardware	Mixed or homogeneous	Dedicated per cluster type
Billing	1 accounting DB	1 accounting DB (shared slurmdbd)
Cross-cluster jobs	N/A	`sbatch -M cluster_name` from shared login
CLI tools	`sacct`, `sbalance`, `stree`, `sqos`	Same commands with `-M` flag

ColdFront Integration

The **SlurmCluster** model supports both architectures. The **root_account** field drives which deployment model is used no code changes needed:

`root_account`	Deployment Model	Affiliations	Example
`root` / `arch` / other	Model A/C	Both JHU + Schmidt	Single cluster or hardware split
`jhu`	Model B	JHU only	Federation -- JHU-dedicated cluster
`schmidt`	Model B	Schmidt only	Federation -- Schmidt-dedicated cluster

```
# Model A: single cluster (both affiliations)
SlurmCluster(name="arch", root_account="root", slurmrestd_url="http://slurmrestd:6820")

# Model B: federation -- one cluster per affiliation
SlurmCluster(name="jhu-cluster", root_account="jhu", slurmrestd_url="http://slurmrestd:6820")
SlurmCluster(name="schmidt-cluster", root_account="schmidt", slurmrestd_url="http://slurmrestd:6820")

# Model C: hardware split (both affiliations per cluster)
SlurmCluster(name="arch-cpu", root_account="arch", slurmrestd_url="http://slurmrestd:6820")
SlurmCluster(name="arch-gpu", root_account="arch", slurmrestd_url="http://restd-2:6820")
```

The **slurm_sync** respects **root_account** automatically:

- Model A/C (**root_account** != jhu/schmidt): creates both **jhu** and **schmidt** root accounts under **cluster_root**, syncs all allocations
- Model B (**root_account** = jhu): creates only **jhu** root, filters out Schmidt PI allocations
- Model B (**root_account** = schmidt): creates only **schmidt** root, filters out JHU PI allocations

Admin presets in the Django Admin pre-fill the correct **root_account** for each deployment model.

Real-time Account Provisioning

When an allocation transitions to Active (staff approval), the system immediately creates the Slurm account hierarchy via REST API no waiting for the periodic sync. This is best-effort: if the REST API is unavailable, the periodic sync (every 3 hours) acts as a safety net.

The provisioning respects Model B affiliation filtering: a JHU PI allocation approved on a Schmidt-only cluster is silently skipped.

Recommendation for JHU (1000 nodes, 10K users): Start with Model A (single cluster with partitions). Migrate to Model B when hardware diversity or operational isolation justifies the complexity. The migration is infrastructure-only add another slurmctld, create a new **SlurmCluster** in Django Admin with the appropriate **root_account**, no code changes needed.

Features

CLOACK = ColdFront + LDAP + OIDC + ARCH + Cluster/Containers + Keycloak

Containerized identity and resource management for HPC clusters at Johns Hopkins University.

Branch Overview

Branch	Version	Purpose	Commits ahead of main
`main`	v0.1	Baseline -- upstream ColdFront + OIDC + LDAP scaffolding	--
`prod`	v0.2	Current production -- LDAP/Slurm sync, staff views, basic signals	69
`dev`	v0.3	Development -- full billing, weekly cap, cluster management, tests	322

Repository: https://github.com/jhu-arch/arch_jhu_schmidt

Documentation Index

File	Description
README.md	ARCH Portal -- stack overview, components, architecture
CLOACK.md	Feature comparison across branches (main vs prod vs dev)
ColdFront	
coldfront/README.md	ColdFront -- HPC resource management portal
coldfront/custom/README.md	Custom configurations (ARCH JHU DSAI Schmidt)
coldfront/BILLING_SETUP_GUIDE.md	Billing system setup guide
coldfront/BILLING_DATA_IMPORT.md	Billing system data import guide
coldfront/BILLING_AND_STORAGE_CHANGES.md	Billing info & storage/backup implementation summary
coldfront/INVOICE_SETTINGS.md	Invoice customization settings
coldfront/SCHOOL_DEPARTMENT_TRACKING.md	School & department tracking for job usage
coldfront/RECOLLECT_HISTORICAL_JOBS.md	Re-collect historical jobs with school & department
coldfront/init/README.md	Initialization scripts (seed data, LDAP import)
Slurm	
slurm/README.md	Slurm -- job scheduler stack, cost enforcement, CLI utilities
slurm/SLURM_RESTART_GUIDE.md	Slurm service restart guide (correct order)
Infrastructure	
openldap/README.md	OpenLDAP -- user & group directory
keycloak/README.md	Keycloak -- identity & access management (OIDC)
keycloak/ldap-providers/README.md	Keycloak LDAP provider snippets and import instructions
postgres/README.md	PostgreSQL -- database server
base/README.md	Base -- shared configuration (UID/GID ranges, paths)
phpldapadmin/README.md	phpLDAPadmin -- LDAP web UI
ondemand/README.md	Open OnDemand -- web portal for HPC
scripts/README.md	Utility scripts (wait-for.sh, etc.)

Feature Comparison: main vs prod vs dev deployment

Billing & Cost Management

Feature	main	prod	dev	Notes
BillingRate catalog (6 rate types)	--	--	Yes	cpu_full, cpu_om, gpu_full, gpu_om, storage, backup
AllocationBilling per allocation	--	--	Yes	IO number, department, billing_type (Pay Per Use / Credit Holder / No Charge)
HardwareCreditPool	--	--	Yes	Track hardware purchases converted to compute credits; credits offset compute, not O&M
SlurmAccountUsagePeriod	--	--	Yes	Per-account per-period usage -- billing source of truth
PaymentStatus (configurable)	--	--	Yes	Admin-managed status codes (pending, paid, overdue, no_charge) without code changes
InvoiceSettings (singleton)	--	--	Yes	Customizable invoice branding, credit/O&M explanation text
Billing Report (staff view)	--	--	Yes	Filterable by PI, project, department, school, month, billing type
PI Billing Summary	--	--	Yes	PI-facing "My Usage & Billing" dashboard
Rate Management UI	--	--	Yes	CRUD for BillingRate with effective date versioning
Hardware Credits UI	--	--	Yes	CRUD for HardwareCreditPool with credit tracking
Unit conversion helpers	--	--	Yes	`dollars_to_grptres_mins()`, `grptres_mins_to_dollars()`, `hours_to_dollars()`
collect_billing command	--	--	Yes	Populate SlurmAccountUsagePeriod from sreport + sacct

Weekly Spend Cap

Feature	main	prod	dev	Notes
Dollar-based weekly cap	--	--	Yes	AllocationAttribute `Weekly Cap ($)`
Account-level GrpTRESMins	--	--	Yes	Cap applied directly on Slurm account (not per-QOS)
Weekly cap usage sync	--	--	Yes	Every 5 min: poll Slurm sreport, store in `Weekly Cap Usage ($)`
Cap-to-TRES conversion	--	--	Yes	`$30/week` -> `GrpTRESMins=billing=1800000` (formula: `$ * 1000 * 60`)
Weekly reset via cron	--	--	Yes	`check_and_run_core_hours_reset` resets GrpTRESMins periodically

How it works:

Project	Cap	Slurm GrpTRESMins
`arochab1_f01_a`	$30/week	`billing=1800000`
`arochab1_f01_b`	$20/week	`billing=1200000`
`bgermro1`	No cap	(not set = unlimited)

Instead of creating per-allocation QOS (**wk_{id}**), the cap is enforced at the account association level:

```
sacctmgr modify account where name=arochab1_f01_a set GrpTRESMins=billing=1800000
```

Storage & Backup

Feature	main	prod	dev	Notes
Project storage/backup fields	--	--	Yes	`storage_enabled`, `storage_tb`, `backup_enabled`, `backup_tb` on Project model
Storage/backup on project forms	--	--	Yes	Create + Update forms with validation
AllocationBilling storage/backup	--	--	Yes	`storage_tb`, `backup_tb` fields for billing

Storage/backup billing rates	--	--	Yes	BillingRate types: `storage` (TB/month), `backup` (TB/month)

Core Hours Reset

Feature	main	prod	dev	Notes
CoreHoursResetSettings	--	--	Yes	Singleton model with 6 period options
Period options	--	--	Yes	weekly, monthly, quarterly, 4-month, semester, annual
`compute_next_reset()`	--	--	Yes	Auto-calculates next reset date based on period
Per-allocation override	--	--	Yes	AllocationAttribute `Next Reset` overrides global schedule
Reset management UI	--	--	Yes	Staff page to configure schedule and trigger manual resets
Next Reset on allocation gauge	--	--	Yes	Shows next reset date below Core Usage gauge

Slurm Integration

Feature	main	prod	dev	Notes
Basic slurm_sync (account + users)	--	Yes	Yes	Create Slurm accounts, add users to accounts
SlurmUserUsageSnapshot	--	Yes	Yes	Per-user usage snapshots from sacct
SlurmCluster / SlurmNode / SlurmPartition models	--	--	Yes	Full cluster topology in Django
Dynamic slurm.conf generation	--	--	Yes	NodeName/PartitionName generated from UI models
TRES billing weights per partition	--	--	Yes	`TRESBillingWeights` field on SlurmPartition
AllowQos per partition	--	--	Yes	`allow_qos` field generates `AllowQos=...` in slurm.conf
SlurmQOS model (DB-driven)	--	--	Yes	QOS definitions per cluster; replaces hardcoded bootstrap
QOS per allocation	--	--	Yes	AllocationBilling.qos FK overrides default QOS assignment
export_qos_config command	--	--	Yes	Generates `qos_config.lua` for cli_filter.lua from DB
Slurm account hierarchy	--	Flat	Yes	`root -> school -> dept -> PI -> project` (auto-managed)
Orphan account cleanup	--	--	Yes	`sync_slurm` removes empty intermediate school/dept accounts
Slurm REST API client	--	--	Yes	`slurmrestclient.py` for slurmrestd integration
Docker-based cluster management	--	--	Yes	`cluster_manager.py` -- build/deploy/restart/stop via UI
sacct collection (every 15 min)	--	--	Yes	SlurmJob model + `collect_sacct` command
scontrol reconfigure via UI	--	--	Yes	"Sync Accounts" button triggers `scontrol reconfigure`
TRES types configuration	--	--	Yes	`SlurmCluster.tres_types` -- configurable per cluster

Department & School Hierarchy

Feature	main	prod	dev	Notes
School model	--	--	Yes	Extracted from Department; FK relationship
Department with school linkage	--	--	Yes	Department -> School -> Slurm hierarchy
AllocationBilling as hierarchy source	--	--	Yes	Billing dept drives Slurm account placement, not Project.department
Department propagation signals	--	--	Yes	Billing dept auto-propagates to Project + PI UserProfile
CSV department import	--	--	Yes	Staff UI for bulk department import
Department user membership	--	--	Yes	FilteredSelectMultiple on Department admin + auto-propagation
Shares on School admin	--	--	Yes	FairShare weights configurable per school

User Management

Feature	main	prod	dev	Notes
Staff scoped views	--	Partial	Yes	prod: basic scoping; dev: full resource-based scoping
AssignStaffResourcesView	--	--	Yes	Superuser assigns cluster resources to staff
CSV user import	--	--	Yes	Bulk user creation with LDAP provisioning

PI SubProject scheme	--	--	Yes	Numeric (A, B, C) or alpha sequence per PI
Activate project (staff)	--	--	Yes	Staff can activate New-status projects
Manage Users view	--	--	Yes	Staff user management across projects
Extra Core Hours attribute	--	--	Yes	Additive core hours on top of base allocation
System account exclusions	--	--	Yes	admin, coldfront, root excluded from all views/sync/billing

Authentication & Identity

Feature	main	prod	dev	Notes
Multi-realm Keycloak OIDC	Partial	Yes	Yes	JhuOIDCBackend + SchmidtOIDCBackend with separate realms
LDAP sync (daily + on-demand)	Partial	Yes	Yes	prod: basic sync; dev: full bidirectional with home dirs
Auto-user creation on OIDC login	--	Yes	Yes	Signal-based user provisioning with auto-affiliation detection
Directory ground truth (UID/GID)	--	--	Yes	Centralized UID/GID allocation, cached on UserProfile, visible in Admin
LDIF migration import	--	--	Yes	Import slapcat dumps preserving UIDs/GIDs/passwords via Import Users UI
PI promotion workflow	Partial	Partial	Yes	dev: enhanced with department propagation
Entra ID integration (JHU)	--	--	Yes	`JHU_ENTRA_ENABLED` flag for Azure AD federation

Scheduled Tasks (django-q)

Task	prod	dev	Schedule
`sync_ldap_all`	Yes	Yes	Daily 2 AM
`sync_slurm_all`	Yes	Yes	Daily 3 AM
`collect_sacct_task`	--	Yes	Every 15 min
`check_and_run_core_hours_reset`	--	Yes	Daily 1 AM
`cleanup_task_queue`	--	Yes	Hourly :30
`sync_weekly_cap_usage`	--	Yes	Every 5 min

All tasks in dev use **hook=TASK_HOOK** for Django Admin LogEntry audit trail.

UI & Admin Improvements

Feature	main	prod	dev	Notes
Sidebar navigation (light theme)	--	--	Yes	Admin Tools, Staff Tools, Clusters, Billing Management
Staff Help page	--	--	Yes	`/staff/help/` -- 21 DB-driven accordion sections editable via Django Admin
Allocation detail enhancements	--	--	Yes	Weekly cap gauge, next reset date, billing context
Job metrics dashboard	--	--	Yes	`/jobs/dashboard/` with per-cluster/partition/user filtering
Cluster list + detail views	--	--	Yes	Full cluster management UI
Inline attribute editing	--	--	Yes	Edit Core Usage, Next Reset, Weekly Cap directly on allocation page
Allocation sync trigger	--	--	Yes	"Sync Now" button on allocation detail
Project creation with storage/backup	--	--	Yes	Storage + backup fields on create/update forms

Infrastructure & DevOps

Feature	main	prod	dev	Notes
`start.sh` orchestration	Basic	Basic	Enhanced	dev: init, start, restart, rebuild, populate, slurm, ldap-reinit, prune-*, test
`start.sh test` command	--	--	Yes	Run Django unit tests inside container
Docker Slurm cluster (dev)	--	--	Yes	Full containerized Slurm: slurmctld, slurmd, slurmdbd, slurmrestd
Unit tests (arch_sync)	--	--	Yes	test_billing, test_models, test_slurm_sync, test_signals
SPANK plugins (C)	--	--	Yes	Cost telemetry compiled into base image

cli_filter.lua (cost estimation)	--	--	Yes	Client-side cost prompt at job submit (login node only)
job_submit.lua (enforcement)	--	--	Yes	Controller-side weekly cap + billing validation
rates.lua (billing rates)	--	--	Yes	Per-partition CPU/GPU rate definitions for Lua scripts
Django-Q audit trail	--	--	Yes	Every async_task uses TASK_HOOK for LogEntry tracking
File-based email (dev)	--	--	Yes	Emails logged to `coldfront/report/emails/` instead of SMTP

What prod has over main (69 commits)

Area	What was added
LDAP sync	Enhanced `ldap_sync.py` -- home directory creation, improved error handling
Slurm sync	Basic `slurm_sync.py` -- account creation, user assignment, usage snapshots
Signals	Auto-user creation on OIDC login, basic allocation signals
Staff views	Initial scoped staff views for project/allocation management
Tasks	Basic django-q scheduled tasks (LDAP sync, Slurm sync)
docker-compose-dev.yml	Dev overlay for local development without host Slurm mounts

What dev has over prod (259 commits)

Area	What was added
Billing	Full billing infrastructure: BillingRate, AllocationBilling, HardwareCreditPool, InvoiceSettings, PaymentStatus, SlurmAccountUsagePeriod
Weekly Spend Cap	Dollar-based caps via account-level GrpTRESMins, 5-min usage sync, spend gauge
Storage/Backup	Project-level fields, form validation, AllocationBilling integration
Core Hours Reset	6-period configurable reset with per-allocation overrides
Slurm Cluster	Full Django models (SlurmCluster/Node/Partition), dynamic slurm.conf, Docker cluster management
QOS	SlurmQOS model (DB-driven), per-allocation override, export_qos_config, AllowQos per partition
Hierarchy	School model, department propagation signals, orphan cleanup
TRES	Configurable tres_types, TRESBillingWeights per partition
CLI	cli_filter.lua, job_submit.lua, rates.lua, SPANK plugins, sbalance
User Management	CSV import, staff resource assignment, SubProject scheme, system account exclusions
UI	Sidebar nav, staff help page, job dashboard, inline editing, billing report, PI billing summary
Testing	Unit tests (billing, models, slurm_sync, signals) + start.sh test command
DevOps	Enhanced start.sh, Docker Slurm cluster, django-q audit trail, file-based email

---

Billing-Free Resources

These resources are never billed enforced in both UI and Slurm CLI:

Resource	Rule
`scavenger` partition	Always free -- preemptable jobs
`ssci-*` accounts	Schmidt Sciences PIs -- always free

Slurm Account Naming Convention

Project Type	Slurm Account	Example
Default PI project	`<pi_username>`	`rdesouz4`
Custom project	`<pi_username>_<abbreviation>`	`rdesouz4_proj1`
SubProject	`<pi_username>_<abbreviation>_<seq>`	`rdesouz4_proj1_A`

Full hierarchy: **root** -> **school** -> **department** -> **pi** -> **project** -> **users**

UID/GID Ranges

Domain	Range
JHU users	10000+
Schmidt users	30000+
Slurm system	450
Munge system	449

Staff Help & Reference Guide

Operational reference for ARCH ColdFront staff. Covers billing, Slurm integration, weekly spend caps, core hours reset, and more.

{{ section.title }}

{{ section.content|safe }}

ColdFront

Django-based allocation portal for ARCH at Johns Hopkins University. Provides user authentication (multi-realm Keycloak OIDC), allocation management, Slurm integration, billing/invoicing, job metrics, and Docker-based cluster orchestration.

Build

```
docker build -t arch/coldfront coldfront/
```

Image: **python:3.10-slim** with PostgreSQL, LDAP, Slurm CLI, Munge, and Docker CLI.

Startup (entrypoint.sh)

1. Deploy custom code from **/opt/coldfront/custom/** into site-packages
2. Run database migrations (default + slurm_jobs)
3. Create superuser (if **DJANGO_SUPERUSER_USERNAME/PASSWORD** set)
4. Seed LDAP users into Django (if **SEED_ON_RESTART=true**)
5. Collect static files
6. Start SSSD (LDAP user resolution)
7. Install cron (nightly backups, periodic Slurm sync)
8. Start Munge (if munge.key present)
9. Start Gunicorn on port 8000

Directory Structure

```
coldfront/
|-- Dockerfile
|-- entrypoint.sh
|-- custom/           -- All JHU/Schmidt-specific code (see custom/README.md)
|   |-- config/       -- Django settings (base, auth, database, plugins)
|   |-- core/user/    -- Extended UserProfile, registration, PI requests
|   |-- external/     -- PI promotion handling
|   |-- etc/          -- Runtime config (coldfront.env, local_settings.py)
|   |-- plugins/
|   |   |-- arch_oidc/   -- Multi-realm Keycloak OIDC backends
|   |   L-- arch_sync/   -- Sync, billing, cluster mgmt, job metrics
|   |-- static/       -- CSS/JS assets
|   L-- templates/    -- HTML overrides
L-- init/             -- Startup scripts (seed_initial_data.py, LDAP sync triggers)
```

Key Config

File	Purpose
`custom/etc/coldfront.env`	Django env vars (loaded by docker-compose)
`custom/etc/local_settings.py`	Runtime overrides (mounted, no rebuild needed)
`custom/config/base.py`	Installed apps, django-q schedule, middleware
`custom/config/auth.py`	LDAP + OIDC authentication backends

Environment Variables

Variable	Purpose
`DJANGO_SUPERUSER_USERNAME`	Auto-create superuser
`DJANGO_SUPERUSER_PASSWORD`	Superuser password
`SEED_ON_RESTART`	`true` = sync LDAP users on every start
`PLUGIN_SLURM`	`1` = enable Slurm plugin
`SLURM_NOOP`	`1` = no-op Slurm mode for testing
`LDAP_ADMIN_PASS`	LDAP bind password (for SSSD)

Port

- 8000 (Gunicorn HTTP)

Development

Custom files are deployed from mounted volume into site-packages on every container start. Code changes take effect on restart without rebuilding:

```
docker restart coldfront
```

Management commands:

```
docker exec coldfront python manage.py migrate
docker exec coldfront python manage.py collect_sacct --current-week
docker exec coldfront python manage.py sync_slurm
docker exec coldfront python manage.py shell
```

Admin Features

- CSV Import Admin > Import Departments / Import Users (sidebar links). Users created with **/bin/false** shell in LDAP
- Core Hours Reset Admin > Billing > Reset Core Hours. Configurable period (weekly/monthly/quarterly), manual trigger, daily django-q check
- Deny Allocation Modal with pre-filled reasons (Billing Info missing, etc.). Saved to **allocation.description**, shown as red alert banner to PI
- SubProject Scheme Set numeric (0,1,2) or alphabetic (A,B,C) per PI/resource. Locked after first SubProject is assigned
- Inline Attribute Editing Staff can edit Core Usage, Extra Core, Next Reset directly on Allocation Detail
- AllocationBilling dept -> PI profile When billing department is set, auto-added to PI's UserProfile.departments

- Service Health & Logs **/user/service-health/** and **/user/service-logs/** open to all authenticated users (works with admin impersonation)

Custom Configurations

This folder contains all customizations for ColdFront specific to the ARCH project at Johns Hopkins University.

Structure

```
custom/
|-- config/          -- Overrides for coldfront.config module
|   |-- auth.py      -- Authentication settings and pipelines
|   |-- base.py      -- Base settings (DATABASE, INSTALLED_APPS, MIDDLEWARE, etc.)
|   |-- core.py      -- Core functionality settings (features, options)
|   |-- database.py  -- Database configuration
|   |-- email.py     -- Email backend settings
|   |-- env.py       -- Environment variable parsing utilities
|   |-- logging.py   -- Logging configuration
|   |-- plugins/     -- Plugin-specific settings (LDAP, Slurm, etc.)
|   |-- settings.py  -- Additional Django settings
|   |-- urls.py      -- URL routing with ARCH custom routes
|   L-- wsgi.py      -- WSGI application entry point
|
|-- core/            -- Overrides for coldfront.core module
|   L-- user/        -- Custom user app
|       |-- models.py    -- Extended UserProfile with has_pi_request field
|       |-- views.py     -- Custom RegisterView, promote_view
|       |-- forms.py     -- Custom registration form
|       |-- admin.py     -- Custom admin configuration
|       |-- signals.py   -- User creation signals
|       |-- urls.py      -- User app routes
|       |-- migrations/  -- Database migrations
|       L-- templates/user/
|           L-- login.html  -- Custom login with JHU/Schmidt realm buttons
|
|-- external/        -- Overrides for coldfront.external module
|   |-- promote.py   -- PI promotion request handling
|   L-- __init__.py
|
|-- etc/             -- Runtime configuration and utilities
|   |-- coldfront.env        -- Environment variables for Django
|   |-- local_settings.py    -- Local Django settings override
|   |-- ldap_sync_mysql_run_it_once.py  -- One-time LDAP -> ColdFront sync
|   L-- README.md            -- Configuration documentation
|
L-- plugins/         -- Custom authentication and sync plugins
    |-- arch_oidc/   -- Multi-realm Keycloak OIDC authentication
    |   |-- backends.py  -- JHU and Schmidt realm OIDC backends
    |   |-- urls.py      -- OIDC login/logout routes
    |   L-- views.py     -- Custom OIDC views
    |
    L-- arch_sync/   -- User synchronization, Slurm, billing, job metrics
        |-- apps.py           -- App configuration
        |-- signals.py        -- Post-create user signals, allocation sync dispatch with admin audit
        |-- tasks.py          -- Async sync tasks with completion hook, admin LogEntry audit trail
        |-- models.py         -- SlurmJob, BillingRate, AllocationBilling, Department, etc.
        |-- admin.py          -- Admin config with weekly cap reset action
        |-- job_views.py      -- Job Usage Metrics dashboard, detail, CSV views
        |-- billing_views.py  -- Billing report, PI dashboard, invoice views
        |-- billing_urls.py   -- URL routing for billing + job metrics
        |-- cluster_views.py  -- Cluster management views (Build Images, Deploy, Restart, Stop)
        |-- cluster_manager.py-- Docker-based Slurm cluster orchestration (images_built, build_images, deplo...
        |-- utils/
        |   |-- slurm_sync.py -- Slurm <-> ColdFront sync (accounts, QOS, caps, usage)
        |   L-- ldap_sync.py  -- LDAP user/group provisioning (adds to realm users group)
        |-- management/commands/
        |   |-- collect_sacct.py          -- Import per-job data from sacct or file
        |   |-- sync_slurm.py             -- Slurm sync management command
        |   |-- populate_fake_billing_data.py -- Dev data seeding
        |   L-- ...
        L-- templates/arch_sync/
            |-- jobs/dashboard.html       -- Job Metrics dashboard template
            |-- jobs/detail.html          -- Raw job list template
            L-- billing/                  -- Billing report, invoice templates
```

How These Customizations Work

1. Config Module Override: The **config/** folder is copied into **site-packages/coldfront/config** during Docker build, overriding upstream settings.

2. Core Module Override: The **core/** folder is copied into **site-packages/coldfront/core**, allowing custom user app to

extend upstream models.

3. External Module Override: The **external/** folder is copied into **site-packages/coldfront/external** for PI promotion handling.

4. Plugin Registration: **arch_oidc** and **arch_sync** are custom plugins registered in **config/base.py** under **EXTRA_APPS**.

5. Local Settings: **etc/local_settings.py** is mounted at runtime to allow configuration changes without rebuild.

6. Environment Variables: **etc/coldfront.env** is sourced by **docker-compose.yml** to configure Django environment.

7. Startup Seed (**coldfront/init/seed_initial_data.py**): Runs at every container start to sync LDAP users into Django, create per-PI projects/allocations, and add allocation members from LDAP groups. Controlled by **SEED_ON_RESTART** in **.env**:

Value	Behavior
`true` (default)	Sync LDAP -> ColdFront on every container start (idempotent)
`false`	Skip LDAP sync on routine restarts -- use when coldfront restarts are frequent and LDAP is stable

Set in **.env** (written by **./start.sh init**). To force a one-time re-sync after LDAP changes, use **./start.sh ldap-reinit** which also removes the OpenLDAP idempotency marker and restarts dependent services.

Key Customizations

Authentication (arch_oidc)

- Supports multiple Keycloak realms: JHU and Schmidt
- Separate OIDC backends for each realm
- Custom login template with dual-button UI (custom/core/user/templates/user/login.html)

User Management (custom/core/user)

- Extended UserProfile with **has_pi_request** field
- Custom registration form with Slurm integration
- PI promotion request handling via custom/external/promote.py

User Sync, Billing & Job Metrics (arch_sync)

- Syncs users from LDAP (OpenLDAP for ARCH pilot); new users added to realm **users** group
- Integrates with Slurm for compute allocation mapping
- Automatic user creation on first OIDC login
- Billing & Invoicing System: Two-track model with hardware credits and O&M charges
- Job Usage Metrics: XDMoD-style dashboard with group-by analytics (User, Account, Partition, QOS, State, Department, School)
- Per-job tracking: **SlurmJob** model populated by **collect_sacct** from sacct output or CSV file
- Weekly spend cap: Account-level **GrpTRESMins=billing** enforcement (dollar cap when billing != cpu)
- Slurm account naming: default = **, project = _**
- Default project protection: Abbreviation and IO Number fields are disabled (read-only) on the edit form for default projects

Billing & Invoicing System

The billing system supports two billing models per allocation:

1. Pay Per Use: Users pay full rate (compute + O&M bundled) for all hours used

2. Hardware Credit Holder: Users who purchased hardware receive credits (prepaid compute hours) that offset usage. O&M charges (power, cooling) are billed separately in cash.

3. No Charge: Allocations with no billing

Key Features:

- Staff Billing Report (**/billing/report/**): Filterable DataTable showing usage by allocation, PI, department, school, or affiliation. Columns: Account, PI, IO#, Jobs, Partition, CPU Hours, GPU Hours, Credits Used, O&M Charges, Total. Download buttons: Summary CSV (PI/IO/Total), IO CSV (by IO number), Detail CSV (per allocation), and formatted PDF report
- Rate Management (**/billing/rates/**): Superuser can set billing rates for CPU/GPU/storage/backup with effective date ranges
- Hardware Credit Pools (**/billing/credits/**): Track hardware purchases converted to compute credit pools with expiry dates
- PI Billing Dashboard (**/billing/my-usage/**): Shows credit balance, O&M charges owed, and credit runway in separate sections (prevents confusion between credits and cash)
- Printable Invoices (**/billing/my-invoice//**): Two-section invoice showing compute credits consumed (non-cash) and O&M charges due (cash). Includes IO/cost center number for payment processing. Supports email sending.
- Allocation Billing Config: Staff can set billing type, IO number, department, and storage/backup TB per allocation via "Billing Info" card on Allocation Detail page
- Department Tracking: Users can belong to multiple departments; managed from Department Admin page (dual-pane filter widget) or UserProfile Admin. When a PI is added to a department, their projects without a department are auto-assigned and Slurm sync is triggered to reparent accounts.
- CSV Import (Admin Tools -> Import Departments / Import Users): Superuser-only views for bulk importing departments and users from CSV files. Department CSV: **name,code,school,school_code,affiliation**. User CSV: **full_name,email,uid**. SSCI users must have **ssci-** prefix; JHU users must use JHED ID with **@jhu.edu**/**@jh.edu**/**@jhmi.edu** email (or **ext-** prefix for non-JHU affiliates). Existing records are skipped. Users are created with **/bin/false** shell in LDAP and activated when a PI adds them to an allocation.
- Usage Collection: **manage.py collect_billing** command collects CPU/GPU hours and job counts from Slurm per billing period

Database Models:

- **BillingRate**: Rate catalog with cluster/partition/type (cpu_full, cpu_om, gpu_full, gpu_om, storage, backup) and effective dates
- **AllocationBilling**: Per-allocation billing config (io_number, billing_type, department, storage/backup TB)
- **HardwareCreditPool**: Hardware purchase records with total credits and credits consumed tracking
- **SlurmAccountUsagePeriod**: Per-period usage snapshot (source of truth for billing: cpu_hours, gpu_hours, job_count)
- **School**: Organizational unit (name, code, affiliation, shares) manages Slurm fairshare at school level
- **Department**: FK to School; organizational filtering and Slurm dept-level fairshare
- **UserProfile.departments** (M2M): Users can belong to multiple departments

Setup:

Dev (all-in-one):

```
./start.sh populate dev --partitions premium,scavenger,gpu
```

This runs in order: **import_departments** -> **populate_fake_billing_data** (rates, **AllocationBilling**, credit pools) ->

generate_slurm_usage_data (**SlurmAccountUsagePeriod**) -> **collect_sacct** **clear (SlurmJob).**

Production / manual steps:

1. Import departments: via Admin Tools -> Import Departments (CSV upload), or **manage.py import_departments departments.csv**

2. Import users: via Admin Tools -> Import Users (CSV upload). Users are created with unusable password + LDAP **/bin/false** shell. Activated when a PI adds them to an allocation (triggers ldap_sync + slurm_sync signals)

3. Assign PIs to departments: **manage.py assign_pi_departments pi_departments.csv**

3. Create billing rates via Django Admin: **/admin/arch_sync/billingrate/add/**

4. When an allocation is approved (status -> Active), **AllocationBilling** is created automatically. If the project has an IO number, **is_billable=True** is set automatically. Otherwise, staff updates billing info via Allocation Detail -> "Billing Info" card.

5. Collect job data: **manage.py collect_sacct current-month**

See **coldfront/templates/examples_departments.csv** and **coldfront/templates/examples_pi_departments.csv** for CSV format examples.

Cluster Management (arch_sync)

Docker-based Slurm clusters are managed from the Cluster Detail page (**/cluster//**). Actions follow a required sequence:

Step	Button	Condition	Action
1	**Build Images**	Always available	Builds all 6 Slurm Docker images (`arch/cn-rockylinux`, `arch/slurm-munge`, `arch/slurmdbd`, `arch/slurmctld`, `arch/slurmd`, `arch/slurmrestd`) -- takes several minutes
2	**Deploy**	Enabled only after images are built	Creates the compose overlay (`docker-compose-slurm-<name>.yml`) and starts all services
3	**Restart**	Only after deploy	Restarts all running Slurm containers
3	**Stop**	Only after deploy	Stops all Slurm containers
--	**Redeploy**	After deploy, replaces Deploy button	Recreates containers with current config (rolling redeploy)

The Deploy button is disabled (grayed out with tooltip) until **images_built()** confirms all required images are present on the Docker host. This prevents the common failure mode where **deploy()** fails silently because **arch/slurmd** or **arch/slurmrestd** images are missing.

cluster_manager.images_built() checks the local Docker daemon for the presence of all 6 required Slurm images. Returns **False** if Docker is unavailable.

ClusterActionView handles all actions via POST to **/cluster//action//**. Long-running operations (**build_images**, **deploy**) run in a background thread to avoid Gunicorn timeout.

Backend Services

- LDAP authentication and user directory
- MariaDB relational database
- Keycloak identity provider with Entra ID integration

Deployment Notes

When deploying, ensure:

1. **coldfront/custom/** folder is included in git history (not in .gitignore)
2. Environment variables from **custom/etc/coldfront.env** are loaded
3. Volume mounts in docker-compose point to correct paths
4. Database migrations are current: **python manage.py migrate**
5. Static files collected: **python manage.py collectstatic**

Billing Setup Guide

This guide walks you through populating example data for the billing system.

Step 1: Import Departments

Create departments from CSV:

```
docker exec coldfront python manage.py import_departments /path/to/examples_departments.csv
```

Or use the example file provided:

```
docker exec coldfront python manage.py import_departments examples_departments.csv --dry-run
docker exec coldfront python manage.py import_departments examples_departments.csv
```

CSV Format: **name,code,school,affiliation**

- Example: **"Computer Science","CS","Whiting School of Engineering","jhu"**

See **examples_departments.csv** for 12 example departments across JHU schools and Schmidt Science.

Step 2: Assign PIs to Departments

Bulk assign PIs to departments from CSV:

```
docker exec coldfront python manage.py assign_pi_departments /path/to/examples_pi_departments.csv --dry-run
docker exec coldfront python manage.py assign_pi_departments /path/to/examples_pi_departments.csv
```

CSV Format: **username,department_name**

- Example: **rdesouz4,"Computer Science"**

See **examples_pi_departments.csv** for a sample file.

Step 3: Create Billing Rates

Automatic (first deploy): Example billing rates are seeded automatically on first startup if none exist. The default rates match **slurm/conf/rates.lua**:

Rate Type	Partition	Rate	Description
CPU Full	(all)	$0.35/SU	Pay-per-use (compute + O&M)
CPU O&M	(all)	$0.10/SU	Credit holders (O&M only)
GPU Full	(all)	$1.20/GPU-hour	Pay-per-use
GPU O&M	(all)	$0.40/GPU-hour	Credit holders
CPU Full	premium	$0.35/SU	Premium partition

CPU O&M	premium	$0.10/SU	Premium O&M
CPU Full	scavenger	$0.10/SU	Scavenger (preemptible)
CPU O&M	scavenger	$0.05/SU	Scavenger O&M
Storage	(all)	$0.025/TB/month	Storage
Backup	(all)	$0.015/TB/month	Backup

Manual: Go to Billing > Manage Rates > Create Rate in the UI, or re-seed by deleting existing rates and restarting:

```
docker exec coldfront python -c "import django; django.setup(); from coldfront.plugins.arch_sync.models imp...
docker restart coldfront
```

Step 4: Configure Allocation Billing

For each allocation you want to bill:

1. Go to Allocation Detail
2. Scroll to Billing Info card
3. Set:
   - IO Number: Cost center (e.g., **1010500000**)
   - Billing Type:
   - **Pay Per Use** charged at full rate
   - **Credit Holder** credits offset compute, O&M charged separately
   - **No Charge** not billed
   - Storage TB: Provisioned storage amount (manual)
   - Backup TB: Provisioned backup amount (manual)
4. Click Update Billing

Step 5: Collect Usage Data

Once allocations have Slurm usage, collect billing data:

```
# Current billing period
docker exec coldfront python manage.py collect_billing --current-period

# Or specific date range
docker exec coldfront python manage.py collect_billing --start 2025-01-01 --end 2025-03-31

# Dry run (preview without saving)
docker exec coldfront python manage.py collect_billing --current-period --dry-run
```

This collects CPU hours, GPU hours, and job counts from Slurm sreport and sacct.

Step 6: View Reports

Staff Views

- Billing Report (filterable): **/billing/report/**
- Summary: total jobs, CPU hours, GPU hours, average
- Filter by: PI, school, department, billing type, date range
- Export: CSV button at bottom

- Invoice List: **/allocation/invoice/**
- All allocations with **is_billable=True**
- Click allocation to see details

- Rate Management: **/billing/rates/**
- View all rates
- Add/edit rates (superuser only)

- Hardware Credits: **/billing/credits/**
- View all credit pools
- Add hardware purchase -> convert to credits

PI Views

- My Usage & Billing: **/billing/my-usage/**
- Left: Credit balance, dollar equivalent, runway estimate
- Right: O&M charges owed (30-day rolling)
- Recent usage table

- My Invoice: **/billing/my-invoice//**
- Printable invoice
- Two sections:

1. Compute Credits Used (hardware-backed, not a cash charge)
2. O&M Charges (cash due)

- Browser print (Ctrl+P) to save as PDF

Example Workflow

1. Import departments:

```
docker exec coldfront python manage.py import_departments examples_departments.csv
   docker exec coldfront python manage.py assign_pi_departments examples_pi_departments.csv
```

2. Create rates:

```
docker exec coldfront python manage.py shell < populate_example_rates.py
```

3. Configure allocations:

- Go to rdesouz4's allocation in ColdFront

- Set IO: **1010500000**
- Set Billing Type: **Pay Per Use**
- Click Update Billing

4. Collect usage:

```
docker exec coldfront python manage.py collect_billing --current-period
```

5. View reports:

- Staff: **/billing/report/** -> filter by department "Computer Science"
- PI (as rdesouz4): **/billing/my-usage/** -> see charges
- PI: **/billing/my-invoice/123/** -> print invoice

Key Concepts

Two-Track Billing Model

- Credits (from hardware purchase)
- Cover compute hours ONLY
- Do NOT pay O&M costs
- UI shows clearly as separate from cash

- O&M Charges (always cash)
- Power, cooling, maintenance
- Applied separately per hour used
- Cannot be paid with credits

Billing Types

Type	CPU Charge	O&M Charge	Uses Credits?
Pay Per Use	Full rate (cpu_full)	Included	No
Credit Holder	$0	O&M rate (cpu_om)	Yes (compute only)
No Charge	$0	$0	No

Troubleshooting

Q: Billing Report shows no data

- Check that allocations have **is_billable=True** in the Billing Info card
- Run **collect_billing** to gather usage data from Slurm

Q: Invoice List is empty

- Verify allocations have Billing Info card configured
- Check that **is_billable=True** is set

Q: PI can't see My Usage & Billing

- Verify they have at least one allocation with billing configured
- Check they're logged in as that PI user

Q: Rates not applying

- Verify **effective_from** date is on or before usage period end date
- Check **effective_to** is null (active) or after usage period

Files Included

- **examples_departments.csv** 12 departments (JHU Whiting, Krieger, Medicine + Schmidt Science)
- **examples_pi_departments.csv** 10 PIs assigned to departments (including rdesouz4)
- **populate_example_rates.py** Create 6 example rates via Django shell
- **BILLING_SETUP_GUIDE.md** This file

Departments Included

Whiting School of Engineering (JHU)

- Computer Science (rdesouz4)
- Electrical & Computer Engineering (agarcia7)
- Applied Physics (mchen2)
- Mechanical Engineering (ekim6)
- Biomedical Engineering

Krieger School of Arts & Sciences (JHU)

- Biology (jsmith5)
- Chemistry (bpatel9)
- Physics (lrodriguez4)

School of Medicine (JHU)

- Neuroscience (kwilson3)
- Molecular Biology & Genetics

Schmidt Science

- Structural Biology (dgonzalez1)
- Computational Biology (hjones8)

Quick Start

```
# 1. Import 12 departments
docker exec coldfront python manage.py import_departments examples_departments.csv

# 2. Assign 10 PIs to departments
docker exec coldfront python manage.py assign_pi_departments examples_pi_departments.csv

# 3. Create 6 billing rates
docker exec coldfront python manage.py shell < populate_example_rates.py

# 4. Configure allocations for billing
#    For each PI with an allocation:
#    - Go to Allocation Detail
#    - Scroll to Billing Info card
#    - Set IO Number (e.g., 1010500000)
#    - Set Billing Type: Pay Per Use or Credit Holder
#    - Click Update Billing

# 5. Collect Slurm usage data
docker exec coldfront python manage.py collect_billing --current-period

# 6. View reports
#    Staff: /billing/report/ (filter by school/department/PI)
#    PI: /billing/my-usage/ (see your charges)
#    Invoice: /allocation/invoice/ (all billable allocations)
```

Test Users (10 PIs across multiple departments):

- rdesouz4 (Computer Science, Whiting)
- jsmith5 (Biology, Krieger)
- mchen2 (Applied Physics, Whiting)
- agarcia7 (ECE, Whiting)
- bpatel9 (Chemistry, Krieger)
- kwilson3 (Neuroscience, Medicine)
- dgonzalez1 (Structural Biology, Schmidt)
- hjones8 (Computational Biology, Schmidt)
- lrodriguez4 (Physics, Krieger)
- ekim6 (Mechanical Engineering, Whiting)

Billing Data Import

This directory contains sample CSV files and instructions for populating the ColdFront billing system with departments, rates, billing configurations, and hardware credit pools.

CSV Files Overview

1. departments.csv

Purpose: Define the organizational structure (schools, departments, affiliations)

Format:

```
name,code,school,affiliation
Computer Science,CS,Whiting School of Engineering,jhu
Data Science,DS,Schmidt Sciences Institute,schmidt
```

Fields:

- **name** (required): Department name (e.g., "Computer Science")
- **code** (optional): Department code (e.g., "CS")
- **school** (optional): School/Institute name (e.g., "Whiting School of Engineering")
- **affiliation** (optional): Either "jhu" or "schmidt" for filtering/organization

Import Command:

```
python manage.py import_departments departments.csv
```

2. pi_departments.csv

Purpose: Assign PIs to departments (supports multiple departments per PI)

Format:

```
username,department_names
testuser1,Computer Science;Applied Physics
testuser2,Data Science
```

Fields:

- **username** (required): JHU username
- **department_names** (required): Semicolon-separated list of department names

Import Command:

```
python manage.py assign_pi_departments pi_departments.csv --dry-run
python manage.py assign_pi_departments pi_departments.csv  # Run without --dry-run after verification
```

3. billing_rates.csv

Purpose: Define billing rates for compute, storage, and backup resources

Format:

```
cluster,partition_name,rate_type,rate_per_unit,effective_from,effective_to,notes
ARCH,gpu,cpu_full,0.45,2026-01-01,,Full rate for CPU hours (compute + O&M bundled)
ARCH,gpu,gpu_om,0.40,2026-01-01,,O&M only charge for GPU hours
```

Fields:

- **cluster** (optional): SLURM cluster name
- **partition_name** (optional): Partition name (empty = all partitions)
- **rate_type** (required): One of:
- **cpu_full**: CPU hours at full rate (compute + O&M)
- **cpu_om**: CPU hours at O&M only (for credit holders)
- **gpu_full**: GPU hours at full rate
- **gpu_om**: GPU hours at O&M only
- **storage**: Storage per TB/month
- **backup**: Backup per TB/month
- **rate_per_unit** (required): Cost per unit (decimal, up to 6 decimals)
- **effective_from** (required): Start date (YYYY-MM-DD)
- **effective_to** (optional): End date or blank for currently active
- **notes** (optional): Description or comments

Import via Django Admin:

Go to **/admin/arch_sync/billingrate/** and add rates manually, or create a management command for bulk import.

4. allocations_billing.csv

Purpose: Configure billing per allocation (IOS code, billing type, storage/backup provisioning)

Format:

```
allocation_id,ios_number,department_name,billing_contact,billing_type,storage_tb,backup_tb,is_billable
1,IO-2024-001,Computer Science,john.doe@jhu.edu,pay_per_use,10.0,5.0,True
2,IO-2024-002,Data Science,jane.smith@ssci.jhu.edu,credit_holder,50.0,25.0,True
```

Fields:

- **allocation_id** (required): ColdFront allocation ID
- **ios_number** (required): IOS/cost center for billing
- **department_name** (required): Department name (must exist)
- **billing_contact** (required): Email or name of billing contact
- **billing_type** (required): One of:
- **pay_per_use**: Full rate billed to user
- **credit_holder**: Hardware credits offset compute; O&M charged separately
- **no_charge**: No billing for this allocation

- **storage_tb** (required): Provisioned storage in TB (0 if none)
- **backup_tb** (required): Provisioned backup in TB (0 if none)
- **is_billable** (required): True/False - whether to include in billing reports

Import via Management Command:

```
python manage.py import_allocation_billing allocations_billing.csv --dry-run
python manage.py import_allocation_billing allocations_billing.csv
```

5. hardware_credits.csv

Purpose: Record hardware purchases and convert to credit pools

Format:

```
allocation_id,description,hardware_value,total_credits,credit_type,granted_date,expiry_date,notes
2,"4x H100 GPU purchase FY2025",85000.00,87600,gpu,2025-01-15,2027-01-15,Purchased through grant funding
```

Fields:

- **allocation_id** (required): ColdFront allocation ID (allocation must exist and have billing config)
- **description** (required): Hardware purchase description
- **hardware_value** (required): Cost of hardware in dollars
- **total_credits** (required): Total compute hours granted
- **credit_type** (required): "cpu" or "gpu"
- **granted_date** (required): Date credits become available (YYYY-MM-DD)
- **expiry_date** (optional): Date credits expire or blank for no expiration
- **notes** (optional): Additional notes

Import via Management Command:

```
python manage.py import_hardware_credits hardware_credits.csv --dry-run
python manage.py import_hardware_credits hardware_credits.csv
```

Quick Start - Populate Fake Data for Testing

To quickly test the billing system without manual CSV setup:

```
# 1. Create fake billing data (departments, rates, allocations, usage)
python manage.py populate_fake_billing_data

# 2. Create allocations and usage specifically for a user
python manage.py populate_user_allocations rdesouz4
python manage.py populate_user_allocations testuser1
```

Then visit:

- **/billing/report/** - See all billing report with fake data
- Click "Print / Save as PDF" to download as PDF

- Click filter by PI and search for the user

Import Workflow

Recommended order:

1. Import departments (creates department records)
2. Assign PIs to departments (links existing users to departments)
3. Import billing rates (sets up rate catalog)
4. Import allocation billing configs (configures each allocation for billing)
5. Import hardware credits (records hardware purchases and credit pools)

Example full workflow:

```
cd /path/to/coldfront

# 1. Import departments
python manage.py import_departments departments.csv

# 2. Assign PIs to departments (with dry-run first)
python manage.py assign_pi_departments pi_departments.csv --dry-run
python manage.py assign_pi_departments pi_departments.csv

# 3. Import billing rates (via admin or custom command)
# (Use Django admin interface at /admin/arch_sync/billingrate/ for now)

# 4. Import allocation billing configs
python manage.py import_allocation_billing allocations_billing.csv --dry-run
python manage.py import_allocation_billing allocations_billing.csv

# 5. Import hardware credits
python manage.py import_hardware_credits hardware_credits.csv --dry-run
python manage.py import_hardware_credits hardware_credits.csv
```

Verification

After importing, verify data in Django admin:

- **/admin/arch_sync/department/** - View departments
- **/admin/arch_sync/billingrate/** - View billing rates
- **/admin/arch_sync/allocationbilling/** - View allocation billing configs
- **/admin/arch_sync/hardwarecreditpool/** - View hardware credit pools

Staff Views

Once data is imported:

- Billing Report: **/billing/report/** - View all charges and credits
- Rate Management: **/billing/rates/** - View/edit rates
- Hardware Credits: **/billing/credits/** - View/manage credit pools
- Allocation Billing: **/billing/allocation//set/** - Configure per-allocation billing

PI Views

Once data is imported, PIs can view:

- My Usage & Billing: **/billing/my-usage/** - Dashboard with credit balance and O&M charges
- My Invoice: **/billing/my-invoice/** - Printable invoice with costs and credit usage

Management Commands

populate_fake_billing_data.py

Creates fake billing data for testing:

```
python manage.py populate_fake_billing_data              # Create fake data
python manage.py populate_fake_billing_data --clear     # Clear and recreate
python manage.py populate_fake_billing_data --no-usage  # Skip usage periods (faster)
```

Creates:

- 7 sample departments (JHU + Schmidt Sciences)
- 8 billing rates (CPU, GPU, storage, backup)
- 5 allocation billing configs
- 15 usage periods with realistic hours
- 1 hardware credit pool

populate_user_allocations.py

Creates allocations and usage for a specific user:

```
python manage.py populate_user_allocations rdesouz4        # Create for rdesouz4
python manage.py populate_user_allocations testuser1       # Create for testuser1
python manage.py populate_user_allocations username --clear # Clear and recreate
```

Creates:

- 1-3 allocations for the user on available resources
- Realistic billing configurations (pay-per-use or credit-holder)
- 3 monthly usage periods with jobs, CPU hours, GPU hours
- Hardware credit pool for credit-holder allocations

Notes

- All CSV imports are idempotent (safe to re-run with same data)
- Use **dry-run** flag to preview changes before committing
- Affiliation is case-sensitive: use "jhu" or "schmidt" (lowercase)
- Credit types: "cpu" for CPU hours, "gpu" for GPU hours
- Rates use up to 6 decimal places for precision (e.g., 0.123456)

- Storage/backup provisioning is manual - update via allocation billing config form
- Use **populate_fake_billing_data** for system-wide testing
- Use **populate_user_allocations USERNAME** for user-specific testing

Billing & Storage Changes

Changes Made

1. Enhanced Billing Information Form

File: **coldfront/custom/plugins/arch_sync/templates/arch_sync/billing/set_billing_form.html**

PIs and admins now have choice interfaces for:

- IO Number: Use project's IO or set a custom one per allocation
- Storage (TB): Use project's storage or set custom per allocation
- Backup (TB): Directly manage backup capacity

Features:

- Shows project defaults as reference
- Radio buttons for easy selection
- Custom fields slide in/out based on choice
- Responsive UI with dynamic visibility

2. Updated Billing Views

File: **coldfront/custom/plugins/arch_sync/billing_views.py**

SetAllocationBillingView changes:

- GET: Now passes **default_storage_tb** to template from project
- POST: Intelligently handles choice logic
- If **io_choice='project'**: uses project's IO number
- If **storage_choice='project'**: uses project's storage
- Otherwise: uses custom values entered by user

Key Logic:

```
# IO Number
if io_choice == 'project':
    io_number = project_io_number
else:
    io_number = request.POST.get('io_number', '').strip()

# Storage
if storage_choice == 'project':
    storage_tb = project_storage_tb
else:
    storage_tb = float(request.POST.get('storage_tb', '0'))
```

3. New Allocation Attribute Type: Backups (TB)

File: **coldfront/custom/plugins/arch_sync/management/commands/create_backup_attribute.py**

Creates a new allocation attribute type allowing:

- PIs/staff to manage backup quotas through allocation attributes
- Consistent with existing Storage Quota management
- Type: Float, changeable, no usage tracking

4. Admin Interface

Already Configured In: **coldfront/custom/plugins/arch_sync/admin.py**

AllocationBillingAdmin fieldsets:

```
Allocation
  |- allocation
  L- department
Billing Configuration
  |- io_number
  |- billing_type
  L- billing_contact
Resources
  |- storage_tb
  L- backup_tb
Status
  L- is_billable
Administration
  |- updated_by
  L- updated_at
```

Deployment Steps

Step 1: Create the Backups Allocation Attribute Type

```
docker compose -f docker-compose-dev.yml exec coldfront \
  python manage.py create_backup_attribute
```

Expected output:

```
v Created allocation attribute type: Backups (TB)
```

Step 2: Verify Changes

The following are now available:

1. For PIs (on their own allocations):

- Navigate to allocation detail page
- Click "Set Billing Info" button
- See choice interfaces for:
- IO Number (project default vs custom)
- Storage (project default vs custom)
- Backup capacity (direct entry)

2. For Staff (in admin panel):
   - Allocation > Set Billing Info
   - View all billing fields with same choice interface
   - Edit allocation attributes to set "Backups (TB)" quota
3. In Admin Dashboard:
   - Go to Allocation Billing
   - View storage_tb and backup_tb in Resources section
   - Filter and search by billing configuration

Data Model

AllocationBilling Model (Updated)

```
allocation (FK)          -> Associated allocation
io_number (CharField)    -> Cost center for billing
billing_type (CharField) -> pay_per_use, credit_holder, no_charge
storage_tb (FloatField)  -> Storage capacity in TB
backup_tb (FloatField)   -> Backup capacity in TB
billing_contact (Text)   -> Contact info
is_billable (Boolean)    -> Include in billing reports
department (FK)          -> Department for attribution
updated_by (FK)          -> Last modified by user
updated_at (DateTime)    -> Last modification timestamp
```

New Allocation Attribute Type

```
Name: Backups (TB)
Type: Float
Changeable: Yes
Has Usage: No
Purpose: Track backup quota per allocation
```

Usage Examples

Example 1: PI Using Project IO

1. Open allocation detail -> "Set Billing Info"
2. IO Number section shows: "Project IO Number: IO-2026-0001"
3. Select "Use project's IO Number"
4. Save -> AllocationBilling.io_number = "IO-2026-0001"

Example 2: PI Overriding Storage

1. Open allocation detail -> "Set Billing Info"
2. Storage section shows: "Project Storage: 100 TB"
3. Select "Use a different storage allocation"

4. Enter custom value: 50
5. Save -> AllocationBilling.storage_tb = 50.0

Example 3: Staff Managing Backups via Attributes

1. Go to Admin -> Allocation Attributes
2. Select attribute type: "Backups (TB)"
3. Enter backup quota: 10 (TB)
4. Also set in AllocationBilling for billing purposes

Verification Checklist

- [ ] Backups (TB) attribute type created
- [ ] Billing form displays IO Number choice interface
- [ ] Billing form displays Storage choice interface
- [ ] Radio buttons toggle custom field visibility
- [ ] Form submits with correct values
- [ ] Admin can see storage_tb and backup_tb in AllocationBilling
- [ ] PI can access and modify billing info (via StaffOrAllocationPIMixin)
- [ ] Database stores custom values correctly

Testing Commands

```
# Create backups attribute
docker compose -f docker-compose-dev.yml exec coldfront \
  python manage.py create_backup_attribute

# Check if attribute was created
docker compose -f docker-compose-dev.yml exec coldfront \
  python manage.py shell -c "
from coldfront.core.allocation.models import AllocationAttributeType
attr = AllocationAttributeType.objects.get(name='Backups (TB)')
print(f'v Found: {attr.name} (type: {attr.attribute_type})')
"

# Test PI access (as PI)
# Navigate to: /allocation/{id}/set-billing-info/
# Should see choice interfaces for IO Number and Storage
```

Files Modified

1. **coldfront/custom/plugins/arch_sync/templates/arch_sync/billing/set_billing_form.html**
   - Added choice interface for IO Number
   - Added choice interface for Storage
   - Added JavaScript for toggling visibility
2. **coldfront/custom/plugins/arch_sync/billing_views.py**

- Updated GET to pass default_storage_tb
- Enhanced POST with choice logic for io_choice and storage_choice

3. **coldfront/custom/plugins/arch_sync/management/commands/create_backup_attribute.py** (NEW)
   - Creates "Backups (TB)" allocation attribute type

Notes

- AllocationBilling model already has storage_tb and backup_tb fields
- Admin interface already displays these fields in Resources fieldset
- StaffOrAllocationPIMixin ensures PIs can only modify their own allocations
- Choice interface gracefully falls back to simple input if no project defaults exist
- Both IO Number and Storage can be managed per-allocation independently

Invoice Settings

The billing system now includes customizable invoice headers and branding options. Superusers can configure how invoices appear to PIs.

How to Customize Invoices

1. Log in as superuser (admin)
2. Go to Django Admin -> **/admin/**
3. Navigate to ARCH Sync -> Invoice Settings
4. Click on the existing settings record (or create one)
5. Customize the following fields:

Available Settings

Organization Information

- Organization Name: Name displayed on invoice header (default: "ARCH | Research Computing Service Center")
- Organization Address: Full address including street, city, state, zip
- Organization Phone: Contact phone number
- Organization Email: Contact email address
- Logo URL: URL to organization logo image (optional)

Invoice Appearance

- Invoice Title: Title displayed at top of invoice (default: "BILLING INVOICE")
- Invoice Footer Text: Text shown at bottom of invoice

Invoice Text & Explanations

- Credit Explanation: Explanation text for hardware credits section
- Default: "Credits represent hardware-backed compute hours purchased by your grant. They cover CPU/GPU runtime only and do NOT offset operational costs (power, cooling, maintenance)."

- O&M Explanation: Explanation text for O&M charges section
- Default: "O&M (Operations & Maintenance) charges cover the operational costs of the facility including power, cooling, and infrastructure maintenance. These charges are real cash costs and cannot be paid with credits."

How It Works

For PIs (Users):

- When a PI views their invoice at **/billing/my-invoice/**
- The invoice displays the customized organization name, address, and logo
- Invoice text explanations help PIs understand credits vs cash charges

For Staff:

- Same customizations appear on staff billing reports

Example Configuration

```
Organization Name: Johns Hopkins University - ARCH Computing
Organization Address: 3400 North Charles Street, Baltimore, MD 21218
Organization Phone: (410) 555-0100
Organization Email: arch-admin@jhu.edu
Logo URL: https://www.jhu.edu/static/images/jhu-logo.png

Invoice Title: MONTHLY BILLING INVOICE

Invoice Footer Text:
"Thank you for using the ARCH Computing Service.
Questions? Email: arch-admin@jhu.edu"

Credit Explanation:
"Your hardware purchase credits represent compute hours on our systems.
These credits cover CPU/GPU runtime hours only.
Operational & Maintenance costs (power, cooling, facility) are billed separately."

O&M Explanation:
"Operations & Maintenance (O&M) charges cover the operational costs of maintaining
the computing facility. These are real cash costs and must be paid separately.
They cannot be covered by hardware credits."
```

Important Notes

- Singleton Model: Only ONE invoice settings record can exist
- Deletion Protection: Settings cannot be deleted
- Updated By: Tracks which user last modified the settings
- Timestamps: Shows when settings were created and last updated
- Logo URL: Must be a valid URL; images are embedded in the invoice HTML

API Access

To get the current settings programmatically:

```
from coldfront.plugins.arch_sync.models import InvoiceSettings

settings = InvoiceSettings.get_settings()
print(settings.organization_name)
print(settings.organization_email)
```

Testing

After customizing invoice settings:

1. Log in as a PI
2. Go to My Invoice -> **/billing/my-invoice/**

3. Verify organization name, address, and logo appear correctly
4. Click Print / Save as PDF to download the invoice
5. Verify all customized text appears in the PDF

School & Department Tracking

Overview

Added school and department information to job tracking (**SlurmJob** model) for comprehensive billing reports and analytics by organizational structure.

Changes Made

1. Database Schema

File: **coldfront/custom/plugins/arch_sync/models.py**

Added two new fields to **SlurmJob** model:

```
school            = models.CharField(max_length=256, blank=True, db_index=True)
department        = models.CharField(max_length=256, blank=True, db_index=True)
```

Denormalization Strategy:

- School and department are stored directly in SlurmJob (denormalized)
- Populated from **Allocation.Project.Department.school** and **Allocation.Project.Department.name**
- This enables faster queries and aggregations without complex joins

2. Migration

File: **coldfront/custom/plugins/arch_sync/migrations/0013_slurmjob_school_department.py**

Creates the two new indexed columns in the **arch_sync_slurmjob** table.

3. Job Collection

File: **coldfront/custom/plugins/arch_sync/management/commands/collect_sacct.py**

Enhanced to populate school and department when collecting jobs:

- Updated **acct_to_alloc** lookup to include school and department info
- For each job, resolves: **account -> allocation -> project -> department -> {name, school}**
- Handles missing allocations gracefully (empty string defaults)

Updated Logic:

```
acct_to_alloc[account] = {
    'allocation_id': alloc.id,
    'school': school_name or '',
    'department': department_name or '',
}
```

4. CSV Export

File: **coldfront/custom/plugins/arch_sync/management/commands/export_jobs_usage_csv.py** (NEW)

New management command to export job data with school and department:

```
python manage.py export_jobs_usage_csv [--start YYYY-MM-DD] [--end YYYY-MM-DD] [--output FILE]
```

Features:

- Exports to **coldfront/templates/jobs_usage.csv** by default
- Includes 33 columns with proper CSV headers
- Includes school and department columns
- Filters by date range (default: last 30 days)

CSV Columns:

```
job_id | job_id_raw | cluster | partition | qos | account | group_name | gid_number |
user | uid_number | submit_time | eligible_time | start_time | end_time |
wall_duration_secs | wait_duration_secs | exit_code | state | n_nodes | n_cpus |
req_cpus | req_mem | req_tres | alloc_tres | time_limit | node_list | job_name |
cpu_time_hours | gpu_time_hours | node_time_hours | school | department | allocation_id
```

5. Job Metrics Dashboard

File: **coldfront/custom/plugins/arch_sync/job_views.py**

Optimized grouping by school and department:

- Changed from Python-side lookup to direct ORM field aggregation
- Much faster queries with **values('school')** instead of account-based lookup
- Enables filtering by school/department at the database level

Updated GROUP_BY_OPTIONS:

```
('department', 'department',  'Department', False),  # direct field
('school',     'school',      'School',     False),  # direct field
```

Deployment Steps

Step 1: Migrate Database

```
docker compose -f docker-compose-dev.yml exec coldfront \
  python manage.py migrate arch_sync
```

Expected output:

```
Applying arch_sync.0013_slurmjob_school_department... OK
```

Step 2: Re-collect Job Data (Optional but Recommended)

To populate school and department for existing jobs:

```
# Re-collect last 30 days of jobs
docker compose -f docker-compose-dev.yml exec coldfront \
  python manage.py collect_sacct --current-month

# Or specify a custom date range
docker compose -f docker-compose-dev.yml exec coldfront \
  python manage.py collect_sacct --start 2026-01-01 --end 2026-03-28
```

Step 3: Export Jobs Usage CSV

```
docker compose -f docker-compose-dev.yml exec coldfront \
  python manage.py export_jobs_usage_csv

# Or with custom date range
docker compose -f docker-compose-dev.yml exec coldfront \
  python manage.py export_jobs_usage_csv --start 2026-03-01 --end 2026-03-28
```

The CSV will be saved to: **coldfront/templates/jobs_usage.csv**

Usage Examples

Example 1: Group Jobs by School

Navigate to **/billing/jobs/** and select "School" from the "Group by" dropdown.

Result: See total CPU hours, job count, and other metrics aggregated by school.

Example 2: Group Jobs by Department

Select "Department" from the "Group by" dropdown.

Result: See metrics aggregated by department across the organization.

Example 3: Export School/Department Data

```
docker compose -f docker-compose-dev.yml exec coldfront \
  python manage.py export_jobs_usage_csv --start 2026-03-01 --output /tmp/jobs_march.csv
```

Then import **/tmp/jobs_march.csv** into your analysis tool (Excel, Python pandas, etc.).

Data Structure Example

Before (Old sacct output)

```
job_id|cluster|account|user|cpu_hours|gpu_hours|...[no school/department]
1001|arch-dev|ricardo|ricardo|2.5|0|...
```

After (New with school/department)

```
job_id|cluster|account|user|cpu_hours|gpu_hours|...|school|department|...
1001|arch-dev|ricardo|ricardo|2.5|0|...|Whiting School of Engineering|Applied Physics|...
```

Query Performance

Before (Python-side lookup)

```
1. Query all accounts: SELECT account FROM arch_sync_slurmjob
2. Lookup account -> department/school: 1000+ Python dict lookups
3. Aggregate in Python: manually sum values
~ O(n) time complexity with manual aggregation
```

After (Direct ORM query)

```
SELECT department, SUM(cpu_time_hours)
FROM arch_sync_slurmjob
WHERE start_time BETWEEN X AND Y
GROUP BY department
~ O(1) database-level aggregation
```

Performance Improvement: 10-100x faster for large datasets.

Verification Checklist

- [ ] Migration applied successfully
- [ ] New columns (school, department) exist in arch_sync_slurmjob table
- [ ] **collect_sacct** populates school/department for new jobs
- [ ] Job Metrics Dashboard groups by School/Department
- [ ] **export_jobs_usage_csv** command works
- [ ] CSV includes school and department columns
- [ ] Existing jobs can be re-collected with school/department info
- [ ] Dashboard aggregation is fast (< 1 second)

Testing Commands

```
# Check if migration is applied
docker compose -f docker-compose-dev.yml exec coldfront \
  python manage.py migrate arch_sync --plan

# Verify new columns exist
docker compose -f docker-compose-dev.yml exec coldfront \
  python manage.py dbshell -c "\d arch_sync_slurmjob" | grep -E "school|department"

# Test collect_sacct with new fields
docker compose -f docker-compose-dev.yml exec coldfront \
  python manage.py collect_sacct --dry-run --current-week

# Verify data in dashboard
# Navigate to: http://localhost/billing/jobs/?group_by=school
# Should show jobs grouped by school with proper aggregations

# Check CSV export
docker compose -f docker-compose-dev.yml exec coldfront \
  python manage.py export_jobs_usage_csv && \
  head -1 /opt/arch/coldfront/templates/jobs_usage.csv
```

Files Modified/Created

File	Type	Change
`coldfront/custom/plugins/arch_sync/models.py`	Modified	Added school, department fields to SlurmJob
`coldfront/custom/plugins/arch_sync/migrations/0013_slurmjob_school_department.py`	Created	Migration for new columns
`coldfront/custom/plugins/arch_sync/management/commands/collect_sacct.py`	Modified	Populate school/department when collecting jobs
`coldfront/custom/plugins/arch_sync/management/commands/export_jobs_usage_csv.py`	Created	New CSV export command with school/department
`coldfront/custom/plugins/arch_sync/job_views.py`	Modified	Optimize department/school grouping with direct ORM

Billing Report Enhancement

The school and department information can now be used for:

1. Billing by Department: Group charges by department for cost allocation
2. School Reports: Generate reports showing each school's total resource usage
3. Cross-organizational Analytics: Compare usage patterns between schools/departments
4. Hierarchical Reporting: JHU Affiliation -> Schools -> Departments -> PIs -> Projects -> Allocations -> Jobs

Notes

- School and department are denormalized in SlurmJob for query performance
- They're populated at job collection time from the allocation's project department
- If a job's allocation doesn't have a department, school/department will be empty
- The fields are indexed for fast grouping/filtering operations
- Existing jobs will have empty school/department unless re-collected

Future Enhancements

- Add affiliation (JHU vs Schmidt) as another denormalized field
- Create a hierarchical tree view of resource usage (JHU -> School -> Department -> PI -> Project)
- Add department-level billing summaries and cost allocation
- Implement department quota/cap tracking

Re-collect Historical Jobs

Overview

Re-collect past job data from Slurm sacct to populate the new school and department fields in SlurmJob records.

Prerequisites

- ? Migration 0013_slurmjob_school_department applied
- ? All allocations have projects with departments assigned
- ? Slurm cluster is accessible (slurmrestd running)

Step-by-Step Process

Step 1: Identify Date Range

Determine which historical period you want to collect:

```
# Example ranges:
# Last 3 months:   2025-12-28 to 2026-03-28
# Last 6 months:   2025-09-28 to 2026-03-28
# All of 2026:     2026-01-01 to 2026-03-28
# Specific month:  2026-03-01 to 2026-03-31

START_DATE="2026-01-01"
END_DATE="2026-03-28"
```

Step 2: Re-collect Job Data with Dry-Run (Optional but Recommended)

First, test the collection without writing to the database:

```
docker compose -f docker-compose-dev.yml exec coldfront \
  python manage.py collect_sacct --start $START_DATE --end $END_DATE --dry-run
```

Expected output:

```
collect_sacct: 2026-01-01 -> 2026-03-28 [DRY-RUN]
Loaded 2847 lines from sacct
collect_sacct done: 2847 parsed, 0 updated -- 2847 raw lines parsed
```

This shows how many jobs would be collected without actually writing them.

Step 3: Actually Collect the Data

Once you're confident, run without **dry-run**:

```
docker compose -f docker-compose-dev.yml exec coldfront \
  python manage.py collect_sacct --start $START_DATE --end $END_DATE
```

Expected output:

```
collect_sacct: 2026-01-01 -> 2026-03-28
Loaded 2847 lines from sacct
collect_sacct done: 2847 new, 0 updated -- 2847 raw lines written
```

Step 4: Verify School & Department Were Populated

```
# Check a sample of jobs
docker compose -f docker-compose-dev.yml exec coldfront \
  python manage.py shell -c "
from coldfront.plugins.arch_sync.models import SlurmJob
jobs = SlurmJob.objects.filter(school__isnull=False).exclude(school='')[:5]
for job in jobs:
    print(f'Job {job.job_id}: {job.account} @ {job.school} / {job.department}')
"
```

Expected output:

```
Job 1001: ricardo @ Whiting School of Engineering / Applied Physics
Job 1002: jasonw @ Whiting School of Engineering / Computer Science
Job 1003: ssci-gomes @ Schmidt Sciences / Biology
...
```

Step 5: Export Jobs Usage CSV

Now export the enriched data to CSV with school and department:

```
docker compose -f docker-compose-dev.yml exec coldfront \
  python manage.py export_jobs_usage_csv --start $START_DATE --end $END_DATE
```

Expected output:

```
Exporting 2847 jobs from 2026-01-01 to 2026-03-28
v Successfully exported 2847 jobs to /opt/arch/coldfront/templates/jobs_usage.csv
```

Step 6: Verify CSV Content

Check that the CSV includes school and department columns:

```
# Show header and first few rows
docker compose -f docker-compose-dev.yml exec coldfront \
  bash -c "head -2 /opt/arch/coldfront/templates/jobs_usage.csv | cut -d',' -f1-15,30-32"
```

Expected output:

```
job_id,job_id_raw,cluster,partition,qos,account,group_name,gid_number,user,uid_number,submit_time,eligible_...
1001,1001,arch-dev,cpu,normal,ricardo,ricardo,1001,ricardo,1001,2026-01-15T10:30:00,2026-01-15T10:31:00,202...
```

Collecting in Batches (Large Datasets)

If you need to collect many months of data, do it in batches to avoid large memory usage:

```
# Collect month by month
for month in {01..03}; do
    START="2026-${month}-01"
    END="2026-${month}-28"
    echo "Collecting $START to $END..."
    docker compose -f docker-compose-dev.yml exec coldfront \
      python manage.py collect_sacct --start $START --end $END
done
```

Troubleshooting

Problem: "No jobs found" / Empty output

Solution: Check that:

1. Slurm sacct has data for that date range: **sacct start=2026-01-01 end=2026-01-31**

2. Jobs exist in the database but without school/department (old jobs)

3. The slurmrestd service is running: **docker compose -f docker-compose-dev.yml ps slurmrestd**

Problem: "school/department are empty"

Solution:

1. Check allocation has a project: **docker compose -f docker-compose-dev.yml exec coldfront python manage.py shell -c "from coldfront.core.allocation.models import Allocation; print(Allocation.objects.count())"**

2. Check project has a department: **docker compose -f docker-compose-dev.yml exec coldfront python manage.py shell -c "from coldfront.core.project.models import Project; print(Project.objects.filter(department*isnull=False).count())"***

3. Re-run collect_sacct - it will populate school/department for all jobs

Problem: "account not found in lookup"

Solution: Jobs with accounts that don't map to allocations will have empty school/department, which is fine. Check logs:

```
docker compose -f docker-compose-dev.yml exec coldfront \
  python manage.py collect_sacct --start 2026-03-01 --end 2026-03-28 2>&1 | grep -i "account\|error"
```

Typical Usage Examples

Example 1: Collect Last Month

```
docker compose -f docker-compose-dev.yml exec coldfront \
  python manage.py collect_sacct --current-month && \
docker compose -f docker-compose-dev.yml exec coldfront \
  python manage.py export_jobs_usage_csv --end $(date +%Y-%m-%d)
```

Example 2: Collect Q1 2026

```
docker compose -f docker-compose-dev.yml exec coldfront \
  python manage.py collect_sacct --start 2026-01-01 --end 2026-03-31 && \
docker compose -f docker-compose-dev.yml exec coldfront \
  python manage.py export_jobs_usage_csv --start 2026-01-01 --end 2026-03-31
```

Example 3: One-Time Historical Collection (Entire Year)

```
# Dry-run first
docker compose -f docker-compose-dev.yml exec coldfront \
  python manage.py collect_sacct --start 2025-01-01 --end 2025-12-31 --dry-run

# Actually collect
docker compose -f docker-compose-dev.yml exec coldfront \
  python manage.py collect_sacct --start 2025-01-01 --end 2025-12-31

# Export
docker compose -f docker-compose-dev.yml exec coldfront \
  python manage.py export_jobs_usage_csv --start 2025-01-01 --end 2025-12-31
```

Performance Tips

- For large date ranges: Collect in monthly batches to reduce memory usage
- For first-time collection: Use **dry-run** first to verify data
- After collection: Run export immediately while data is fresh in cache
- Parallel collection: You can run multiple collect_sacct commands on different date ranges simultaneously in different shells

Verification After Collection

Dashboard Check

Navigate to **/billing/jobs/** and verify:

- ? "Group by: School" shows jobs grouped by school name
- ? "Group by: Department" shows jobs grouped by department name
- ? CPU hours are aggregated correctly by school/department
- ? Load time is fast (< 1 second)

Database Check

```
# Count jobs with school/department populated
docker compose -f docker-compose-dev.yml exec coldfront \
  python manage.py shell -c "
from coldfront.plugins.arch_sync.models import SlurmJob
total = SlurmJob.objects.count()
with_dept = SlurmJob.objects.exclude(school='').count()
print(f'Total jobs: {total}')
print(f'Jobs with school: {with_dept}')
print(f'Coverage: {with_dept/total*100:.1f}%')
"
```

CSV Check

```
# Verify CSV has data and includes school/department
docker compose -f docker-compose-dev.yml exec coldfront \
  bash -c "
echo 'Total rows:' && wc -l /opt/arch/coldfront/templates/jobs_usage.csv
echo 'Rows with school data:' && grep -v '^[^,]*,[^,]*,[^,]*,[^,]*,[^,]*,[^,]*,[^,]*,[^,]*,[^,]*,[^,]*,[^,]...
"
```

Next Steps

After re-collecting historical jobs with school/department:

1. Use in Billing Reports: View billing aggregated by school/department
2. Create Department Hierarchies: Build cost allocation reports
3. Export for Analysis: Use jobs_usage.csv in Excel or Python for analysis
4. Set Departmental Quotas: Use school/department data to set resource limits

Schedule Regular Collection

To keep jobs_usage.csv up-to-date, run collection daily:

```
# Add to crontab (runs daily at 2 AM)
0 2 * * * docker compose -f docker-compose-dev.yml exec coldfront \
  python manage.py collect_sacct --current-week && \
  python manage.py export_jobs_usage_csv --end $(date +\%Y-\%m-\%d)
```

Or add to Django-Q task scheduler for automatic execution.

Initialization Scripts

test_ldap_provision.py

Smoke-test for **provision_web_user**. Verifies that a newly registered user would be (or is) correctly propagated to LDAP.

Usage inside the container

```
# Dry-run -- reads LDAP, prints what it WOULD do, no writes:
docker exec coldfront python /opt/coldfront/init/test_ldap_provision.py

# Actually create the test user (JHU tree):
docker exec coldfront python /opt/coldfront/init/test_ldap_provision.py --create

# Schmidt tree with a custom username:
docker exec coldfront python /opt/coldfront/init/test_ldap_provision.py \
  --username ssci-test --affiliation schmidt --create
```

Dry-run output

```
[DRY-RUN] provision_web_user for 'test-ldap-user' in tree='jhu'

[DRY-RUN] Would create LDAP entries:
  User DN  : uid=test-ldap-user,ou=People,ou=jhu,dc=arch,dc=cluster
  uidNumber: 10001
  gidNumber: 10000  (users group fallback)
  givenName: Test
  sn       : User
  mail     : test@example.com
  Group DN : cn=test-ldap-user,ou=Groups,ou=jhu,dc=arch,dc=cluster  gidNumber=10001
  memberUid: add test-ldap-user -> cn=users,ou=Groups,ou=jhu,dc=arch,dc=cluster
```

With **create** it writes to LDAP and then verifies the user can be found with an **ldapsearch**.

Slurm

Docker-based Slurm cluster for ARCH. Each subdirectory is a service image built from a shared base.

Sub-Services

Subdirectory	Image	Purpose
`base/`	`arch/cn-rockylinux`	Base image (Rocky 9 + Slurm + Munge + SSSD + SPANK plugins)
`slurmctld/`	`arch/slurmctld`	Controller daemon -- job scheduling, `job_submit.lua` enforcement
`slurmd/`	`arch/slurmd`	Worker + login node -- also serves SSH when `NODE_TYPE=login`
`slurmdbd/`	`arch/slurmdbd`	Accounting daemon -- stores job records in MariaDB
`slurmrestd/`	`arch/slurmrestd`	REST API -- used by bare-metal clusters (JWT + UNIX socket auth); Docker dev uses docker-exec instead
`munge/`	`arch/slurm-munge`	Key init (one-shot) -- generates munge.key, then exits
`conf/`	--	Shared config: `slurm.conf`, `cli_filter.lua`, `rates.lua`, `job_submit.lua`
`spank-plugins/`	--	SPANK plugin C sources (compiled into `.so` in base image)

Build Order

All images inherit from **arch/cn-rockylinux**. Build context must be **slurm/**:

```
docker build -t arch/cn-rockylinux -f slurm/base/Dockerfile slurm/
docker build -t arch/slurm-munge  -f slurm/munge/Dockerfile slurm/
docker build -t arch/slurmctld    -f slurm/slurmctld/Dockerfile slurm/
docker build -t arch/slurmd       -f slurm/slurmd/Dockerfile slurm/
docker build -t arch/slurmdbd     -f slurm/slurmdbd/Dockerfile slurm/
docker build -t arch/slurmrestd   -f slurm/slurmrestd/Dockerfile slurm/
```

Or use the Build Images button in the ColdFront cluster UI.

Ports

Service	Port	Purpose
slurmctld	6817	Controller communication
slurmd	6818	Worker communication
slurmd (login)	22	SSH user access
slurmrestd	6820	REST API (HTTP)
slurmrestd	UNIX socket	Local auth (ColdFront via SO_PEERCRED)

Communication Architecture

ColdFront uses two backends depending on deployment mode:

Docker Dev Mode (default)

```
+-------------+     docker exec          +-------------+
  |  ColdFront  |------------------------> |  slurmctld  |
  |  (qcluster) |  sacctmgr/sinfo/sacct    |             |
  L------+------+                          L------+------+
         |                                        |
         | Docker socket                          | Slurm protocol
         | /var/run/docker.sock                   | (munge auth)
         |                                        |
  +------+------+                          +------+------+
  |   Docker    |                          |  slurmdbd   |--> MariaDB
  |   Daemon    |                          L-------------+
  L-------------+
```

Docker clusters (**use_docker_exec=True**) bypass slurmrestd entirely. ColdFront runs Slurm CLI commands (sacctmgr, sinfo, sacct, sreport) inside the slurmctld container via Docker SDK.

Bare-Metal / Production Mode

```
+-------------+       (HTTP :6820)       +-------------+
  |  ColdFront  |------------------------->|  slurmrestd |
  |  (qcluster) |  REST API v0.0.42        |             |
  L-------------+                          L------+------+
                                                  | Slurm protocol
                                                  | (munge auth)
                                           +------+------+
                                           |  slurmctld  |
                                           L------+------+
                                                  |
                                           +------+------+
                                           |  slurmdbd   |--> MariaDB
                                           L-------------+
```

Bare-metal clusters use slurmrestd REST API with UNIX socket (**SO_PEERCRED**) or JWT auth.

Communication Channels

Channel	From -> To	Protocol	Auth	When Used
Docker exec	ColdFront -> slurmctld container	`docker exec` CLI	Docker socket (Peer UID)	Docker dev mode -- all Slurm queries (accounts, jobs, nodes, partitions)
REST API	ColdFront -> slurmrestd	HTTP (`http://slurmrestd:6820`)	`rest_auth/local` via UNIX socket (`SO_PEERCRED`)	Bare-metal / production clusters
Slurm protocol	slurmrestd -> slurmctld/slurmdbd	Slurm RPC	Munge (shared key via `munge-key` volume)	Translates REST calls to native Slurm operations
Docker API	ColdFront -> Docker daemon	UNIX socket (`/var/run/docker.sock`)	Peer UID	Build images, deploy/restart/stop Slurm containers, read logs, health checks

Key volumes shared between services:

- **munge-key** munge authentication key (all Slurm services)
- **slurm-socket** UNIX socket for slurmrestd local auth (ColdFront + slurmrestd)
- **slurm-state** slurmctld state directory (persistent across restarts)
- **slurm-db-data** MariaDB data (slurmdbd accounting)

Cost Enforcement & UX Plugins

Configuration and plugin code for cost estimation, user confirmation, and runtime cost handling in ARCH Slurm clusters.

Components

1. conf/cli_filter.lua -- Login node, UX only

- Loads billing rates from **conf/rates.lua** at submit time.
- Estimates job cost based on **partition**, CPUs, memory, GPUs, and wall time.
- Weekly budget: if the account has a **wk_*** QOS cap, shows remaining budget and warns if job cost exceeds it.**
- Free partitions/accounts: shows $0 cost estimate with resource details, then proceeds without confirmation. Applies to scavenger partition and **ssci-*** accounts.**
- Billable partitions: shows estimated cost and prompts **[y/N]** before submitting.
- QOS pre-check: verifies user's QOS is allowed on the target partition before submitting. Blocks with a clear error message (e.g., "Your QOS 'scavenger' is not allowed on partition 'premium'") and suggests creating a project with IO Number.
- Allocation-aware: skips billing display when running inside an existing allocation (**SLURM_JOB_ID** set), preventing duplicate output from **srun** within **salloc** (e.g., **interact** sessions).
- Non-interactive (**sbatch**): auto-accept with **export=ALL,ACCEPT_COST=1** or **comment=accept_cost**.
- Slurm's native **AccountingStorageEnforce=associations,limits,qos** provides server-side enforcement as a second layer.

CLI Examples

```
rdesouz4@login01:~$ srun -p scavenger --pty bash

[BILLING] Cost Rate  : $0.1000/SU
          Partition : scavenger
          CPU hours : 1.00 @ Estimated cost: ~$0.0000
[BILLING] Partition 'scavenger' -- no charge.
rdesouz4@cn-03:~$
```

```
rdesouz4@login01:~$ srun -p premium -A ssci-jdoe --pty bash

[BILLING] Cost Rate  : $0.3500/SU
          Partition : premium
          CPU hours : 1.00 @ Estimated cost: ~$0.0000
[BILLING] Account 'ssci-jdoe' -- no charge.
rdesouz4@cn-01:~$
```

```
rdesouz4@login01:~$ srun -p premium -A rdesouz4_pfp -t 120 --pty bash

[BILLING] Cost Rate  : $0.3500/SU
          Partition : premium
          CPU hours : 2.00 @ Estimated cost: ~$0.7000
[BILLING] Proceed? [y/N]: y
[BILLING] Job 31 submitted -- estimated ~$0.7000.
rdesouz4@cn-01:~$
```

Non-interactive sbatch:

```
sbatch --export=ALL,ACCEPT_COST=1 myjob.sh
```

2. conf/rates.lua -- Billing rates table

- Exported from the ColdFront Billing Rates page.
- Keyed by partition (**cpu**, **gpu**) and tier (**full_rate**, **om_only**).
- Loaded by **cli_filter.lua** via **dofile("/etc/slurm/rates.lua")**.
- Update this file when rates change no image rebuild needed (bind-mounted).

3. conf/job_submit.lua -- Controller, enforcement

- Recalculates and enforces cost acceptance on the slurmctld controller.
- Rejects jobs that lack explicit cost acceptance.
- Injects **SLURM_JOB_COST_ESTIMATE** into the job environment.

4. spank-plugins/myspank.c -- SPANK plugin (sample)

- Minimal SPANK plugin that logs a message when loaded on compute nodes.
- Compiled by **base/Dockerfile** -> **/usr/lib64/slurm/myspank.so**.
- Loaded via **conf/plugstack.conf**.

5. conf/spank_reemit_cost.c -- SPANK plugin (optional, runtime)

- Logs/processes **SLURM_JOB_COST_ESTIMATE** at job start on compute nodes.
- Compiled by **base/Dockerfile** -> **/usr/lib64/slurm/spank_reemit_cost.so**.
- Loaded via **conf/plugstack.conf**.
- Can be extended for telemetry or external integrations.

How It Works

Component	Where?	Purpose	Enforces?
`cli_filter.lua`	Login node	Estimate & display cost, weekly budget, UX	No
`rates.lua`	Login node	Billing rates table (loaded by cli_filter)	No
`job_submit.lua`	Controller	Enforce acceptance, inject cost env var	Yes
`myspank.so`	Compute node	Sample SPANK plugin (logs on load)	No
`spank_reemit_cost`	Compute node	Log/process cost at job start	No

Build

SPANK plugins are compiled automatically during the Docker image build.

Build context must be **slurm/** (not **slurm/base/**) so both source paths resolve:

```
docker build \
  --file slurm/base/Dockerfile \
  -t arch/cn-rockylinux \
  slurm/
```

cluster_manager.py (**build_images()**) does this automatically.

---

slurm.conf directives

```
AccountingStorageEnforce=associations,limits,qos
PlugStackConfig=/etc/slurm/plugstack.conf
```

- **AccountingStorageEnforce**: Users without a Slurm account association cannot submit jobs. Also enforces GrpTRESMins limits and QOS constraints.
- Note: **CliFilterPlugins=cli_filter/lua** is NOT in **slurm.conf**. It is injected at runtime by the login node entrypoint only (see **slurmd/entrypoint.sh**). Adding it to **slurm.conf** would break **sbatch** inside slurmctld.

---

plugstack.conf

Plugins are loaded directly (no **plugstack.conf.d** subdirectory):

```
optional /usr/lib64/slurm/myspank.so
optional /usr/lib64/slurm/spank_reemit_cost.so
```

optional means the job proceeds even if the plugin fails to load.

Use **required** to abort the job on load failure.

Volume mounts (per node container)

The following files from **slurm/conf/** are bind-mounted into **/etc/slurm/** at runtime:

Host path	Container path
`conf/slurm.conf`	`/etc/slurm/slurm.conf`
`conf/cgroup.conf`	`/etc/slurm/cgroup.conf`
`conf/cli_filter.lua`	`/etc/slurm/cli_filter.lua`

`conf/rates.lua`	`/etc/slurm/rates.lua`
`conf/plugstack.conf`	`/etc/slurm/plugstack.conf`

Managed by **cluster_manager.py** -> **generate_compose_overlay()**.

CLI Utilities (installed in base image)

All ARCH CLI tools are installed at **/usr/local/bin/** and support **-M** for multi-cluster queries.

User & PI Tools

Command	Purpose
`interact -l`	List available partitions, your accounts & QOS
`interact -p <part> [opts]`	Start interactive session (validates walltime, shows weekly cap budget)
`stree`	Show your own account tree (auto-detects `$USER`)
`stree --full`	Your tree with user associations
`sbalance`	Show account core-hour utilization (current period)
`sbalance --week`	Show weekly spend cap status per account (cap vs usage in $)
`sqos all`	Show all QOS definitions (priority, limits)
`sqos <user>`	Show QOS for a specific user
`sqme`	Show your active jobs (formatted `squeue`)
`seff <jobid>`	Show CPU and memory efficiency for a completed job
`sfeatures`	Show node features, GRES, and partition info
`sbrief`	Job state summary (count by Running, Pending, etc.)

Staff / Admin Tools

Command	Purpose
`stree -r root`	Full hierarchy from root (admin only)
`sassoc`	Show your associations (account, QOS, fairshare)
`sassoc <user>`	Show associations for a specific user
`sassoc --clusters`	List all registered Slurm clusters
`sadmin priority <jobid>`	Set high priority (admin only)
`sadmin addtime <jobid> <time>`	Add time to running job (admin only)
`sadmin settime <jobid> <time>`	Replace time limit (admin only)

```
Note: Old tool names (`slurmtree`, `slurmqos`, `slurmassoc`) still work as symlinks.
```

Account Hierarchy

The Slurm account tree mirrors the ColdFront organizational structure:

```
root
|-- jhu [Fairshare=20]
|   L-- <school> (e.g. it)
|       L-- <department> (e.g. arch)
|           L-- <pi_username>
|               L-- users
L-- schmidt [Fairshare=80]
    L-- ss (Schmidt Sciences)
        L-- ssci-<pi_username>
            L-- users
```

Department placement is driven by **AllocationBilling.department** (not **Project.department**).

Priority: non-default allocation billing dept > default allocation billing dept > Project dept > fallback.

Default departments ensure no PI is ever flat under root:

- JHU PIs without department -> **noschool/nodep**
- Schmidt PIs -> **ss** (Schmidt Sciences)
- Created automatically on startup via **seed_initial_data.py**

System accounts (**admin**, **coldfront**, **root**) are excluded from the Slurm hierarchy.

Sudo (login & compute nodes)

Members of the LDAP group **cn=admin,ou=Groups,ou=jhu,dc=arch,dc=cluster** get passwordless sudo on all nodes. This is written by the entrypoint at container start (SSSD-independent):

```
/etc/sudoers.d/admin:
    %admin  ALL=(ALL)  NOPASSWD: ALL
```

SSSD resolves the LDAP group as Unix group **admin** via **coldfront/sssd.conf** (mounted as **/etc/sssd/sssd.conf.template** and started by the entrypoint).

Portal Setup (Before First Use)

Before PIs can create projects and submit jobs, staff must complete these one-time setup steps in ColdFront:

Step	Where	What
1. Import Schools & Departments	Admin Tools > Import	CSV: `name, code, school, school_code, affiliation`
2. Create Slurm Cluster	Django Admin > Slurm Clusters	Name must match Slurm cluster (e.g. `arch`), set TRES types
3. Create Partitions & Nodes	Django Admin > Slurm Partitions	`scavenger`, `premium`, etc. Set AllowQos, TRESBillingWeights
4. Configure QOS	Django Admin > Slurm QOS	`scavenger` (is_free=True), `normal`, `premium` (auto-created on deploy)
5. Set Billing Rates	Django Admin > Billing Rates	`cpu_full`, `cpu_om`, `gpu_full`, `gpu_om`, `storage`, `backup` with effective dates

6. Activate Terms of Service	Admin Tools > Import ToS	Upload ToS CSV or create in Django Admin. Set `is_active=True`
7. Create ColdFront Resource	Django Admin > Resources	Resource PIs select when requesting allocations

ToS Enforcement

- Without an active ToS, all users can create projects freely.
- With an active ToS, users must accept all required sections before creating a project.
- Users who have not accepted the ToS will not get a Slurm account even if they can log in via Keycloak/LDAP, they cannot submit jobs (**slurm_sync** skips them).

Default vs Paid Allocations

Type	Project Name	Slurm Account	QOS	Billed?
Default	PI username (no abbreviation)	`pi_username`	scavenger	No
Paid	Custom (with abbreviation)	`pi_username_proj1`	normal, premium	Yes

PI Promotion Pipeline

When a staff member promotes a user to PI via the ColdFront staff toggle (**/user/staff-manage-users//toggle-pi/**), the following steps happen automatically:

1. Project created with status **Active** no separate approval step required.
2. Allocation created with status **Active** on every allocatable Cluster resource immediately usable.
3. **run_slurm_sync** is queued (via django-q or inline) scoped to the new PI's username propagates the Slurm account immediately.

The Slurm account name follows these rules:

Project abbreviation	Slurm account name
`default` (none set)	`<pi_username>`
custom (e.g. `proj1`)	`<pi_username>_proj1`
custom + SubProject	`<pi_username>_proj1_A`

The same pipeline runs when using the legacy **promote.py** view (**/user/promote//**).

Usage Gauges

Each allocation has its own Core Usage gauge. The gauge data source depends on the allocation type:

Gauge	Allocation Type	Data Source	What It Shows
Core Usage (Hours)	Default (scavenger)	`slurm2coldfront()` -> SlurmJob table	Free core-hours used (scavenger QOS)
Core Usage (Hours)	Paid (custom project)	`slurm2coldfront()` -> SlurmJob table	Paid core-hours used (normal/premium QOS)
Weekly Cap Usage ($)	Paid with cap set	`sync_weekly_cap_usage()` -> sreport	Weekly dollar spend vs cap

- Default allocations (project = PI username, no abbreviation) get scavenger QOS and are never billed.
- Paid allocations (custom project with abbreviation) get normal/premium QOS and are billed.
- Each allocation has its own Slurm account (**pi_username** vs **pi_username_proj1**), so jobs never mix between gauges.
- The Weekly Cap gauge only appears when **Weekly Cap ($)** or **Weekly Cap (Hours)** is set on the allocation.

Weekly Spend Cap

Allocations can have a Weekly Cap ($) or Weekly Cap (Hours) attribute. Dollar cap takes precedence. The sync pipeline creates a per-allocation QOS (**wk_{alloc_id}**) with **GrpTRESMins=billing=** limits that work with TRESBillingWeights.

Conversion formula

- 1 SU (Service Unit) = $0.001
- **GrpTRESMins = $X * 1,000 * 60** (dollars -> SU -> SU-minutes)
- Example: $50 weekly cap -> **GrpTRESMins=billing=3000000**

How it works

1. AllocationAttribute **Weekly Cap ($)** or **Weekly Cap (Hours)** is set via ColdFront Allocation Detail.
2. **run_slurm_sync** reads the cap value and sets **GrpTRESMins** on the account association:
   - With cap: **GrpTRESMins=cpu={core_hours*60},billing={cap_mins}** (billing differs from cpu)
   - Without cap: **GrpTRESMins=cpu={mins},billing={mins}** (billing = cpu, no separate cap)
3. The cap applies across all partitions automatically (no AllowQos changes needed).
4. When the billing-TRES cap is reached, Slurm rejects new jobs for that account.
5. **sync_weekly_cap_usage** (every 5 min) syncs current usage via **sreport** into an AllocationAttribute for the spend gauge.
6. **cli_filter.lua** detects weekly cap (billing != cpu) and shows cap after estimated cost.
7. **sbalance week** shows weekly cap status per account (color-coded CLI report).

Note: The old **wk_*** QOS approach was abandoned because QOS needed AllowQos on each partition and was not applied automatically as defaultqos.**

GrpTRESMins practical examples

Given: Core Usage (Hours) = 20,000 -> **mins = 20000 x 60 = 1,200,000**

Weekly Cap ($) = $30 -> **cap_mins = $30 x 1000 x 60 = 1,800,000**

`tres_types` (admin)	Weekly Cap	GrpTRESMins result
`cpu,billing`	none	`cpu=1200000,billing=1200000`
`cpu,billing`	$30	`cpu=1200000,billing=1800000`
`cpu,gres/gpu,billing`	none	`cpu=1200000,gres/gpu=1200000,billing=1200000`
`cpu,gres/gpu,billing`	$30	`cpu=1200000,gres/gpu=1200000,billing=1800000`

- All non-billing TRES get the same value: **core_hours x 60**

- **billing** TRES gets the weekly cap value when set; otherwise same as **cpu**
- Cap detection: **billing != cpu** means a weekly cap is active
- The cap applies across ALL partitions (no per-partition cap)

QOS bootstrap

slurm_sync automatically creates **premium** and **scavenger** QOS in Slurm if they don't exist (only **normal** exists by default). Default-project accounts get **qos=scavenger**; custom-project accounts get **qos=normal,premium**.

Partition access control

SlurmPartition.allow_qos generates the **AllowQos=...** directive in slurm.conf. Partitions use **AllowQOS** to restrict which accounts can submit:

Partition	AllowQOS	Effect
`cpu`	`normal,premium`	Only accounts with `normal` or `premium` QOS
`cpu-dev`	`normal,premium`	Same -- excludes scavenger-only accounts
`cpu-dev2`	`normal,premium,scavenger`	Open to all
`scavenger`	`scavenger,normal,premium`	Open to all

Default-project accounts (QOS=**scavenger** only) are restricted to **cpu-dev2** and **scavenger** partitions.

TRES (Trackable RESources)

TRES are the resource types that Slurm tracks, limits, and bills per job. Available TRES are configured in **slurm.conf** via **AccountingStorageTRES**.

TRES	What it is
`cpu`	CPU cores x time
`mem`	Memory allocated
`billing`	Weighted cost metric (calculated from TRESBillingWeights)
`node`	Whole nodes used
`gres/gpu`	GPU devices x time (requires `GresTypes=gpu` + `gres.conf`)

GrpTRESMins on Slurm accounts caps the maximum each group can consume. **slurm_sync** sets this from the allocation's Core Hours. The TRES types included in the limit are configured via **SlurmCluster.tres_types** in the admin (default: **cpu,billing**; production with GPUs: **cpu,gres/gpu,billing**).

TRESBillingWeights on partitions defines how resources are weighted for billing. Configurable per partition via **SlurmPartition.tres_billing_weights** in the admin. Example:

```
CPU=1.0,Mem=0.25G,GRES/gpu=2.0
```

This means 1 GPU counts as 2 "billing units". A job using 1 GPU for 1 hour costs 2 billing-hours. The **billing** TRES in **GrpTRESMins** caps the weighted total, not just raw CPU.

To check available TRES: **sacctmgr show tres format=Type,Name -P -n**

tres_types vs TRESBillingWeights vs BillingRate

These three concepts work together but serve different purposes:

Concept	Where defined	What it does	Example
`tres_types`	`SlurmCluster` admin	Which TRES are included in `GrpTRESMins` limits	`cpu,billing` or `cpu,gres/gpu,billing`
`TRESBillingWeights`	`SlurmPartition` admin	How resources are weighted for the Slurm `billing` TRES	`CPU=1.0,Mem=0.25G,GRES/gpu=2.0`
`BillingRate`	ColdFront Billing Rates	Price in dollars charged to PIs for invoicing	`cpu_full: $0.35/SU`, `gpu_full: $2.50/hr`

tres_types controls what goes into **GrpTRESMins** (Slurm enforcement). Staff sets it once per cluster. Adding **gres/gpu** here makes Slurm track GPU limits.

TRESBillingWeights controls how Slurm calculates the **billing** TRES per job. Set per partition. A GPU partition with **GRES/gpu=2.0** means 1 GPU-hour = 2 billing-units. This determines how fast a job consumes the **GrpTRESMins=billing=** limit. Partition type (CPU vs GPU) is inferred from this field: if it contains **GRES/gpu**, it's a GPU partition.

BillingRate is purely financial the dollar amount shown on invoices and cost estimates. These are independent of Slurm weights and can change each semester without affecting Slurm limits.

They are deliberately kept separate because they change at different rates and serve different audiences (Slurm admin vs finance).

Admin reset

Staff can reset a cap's usage counter via Django Admin -> AllocationAttribute -> select "Weekly Cap ($)" or "Weekly Cap (Hours)" attributes -> action "Reset weekly cap QOS usage counter in Slurm". This deletes and re-creates the QOS to zero the accumulated usage. Supports both dollar and hours cap types.

Full Detail Invoice (per-job breakdown)

The billing report table (**/billing/report/**) has three action buttons per row: Admin, PDF, and TXT. A fourth button Detail will be added to provide a full per-job breakdown while keeping the existing PDF and TXT invoices unchanged.

What the Detail download includes

Section	Content
BILLING DETAILS	Project, PI, IO Number, Department, Billing Type
USAGE SUMMARY	Jobs, CPU Hours, GPU Hours, Storage (TB), Backup (TB)
JOB DETAIL	One row per `SlurmJob` record: JobID, Partition, Start, CPUs, GPUs, CPU-Hrs, GPU-Hrs, State
CHARGES	CPU, GPU, Storage, Backup charges + TOTAL

- Output format: plain text (**.txt** download), 80-char width.
- Capped at 500 job rows; shows "... and N more jobs" if truncated.
- Storage and Backup rows appear only when > 0.

Access control

Role	Can see Detail button?
Superuser / Staff	Yes -- all periods
PI (project owner)	Yes -- own allocation periods only
Regular user	No

Implementation plan

- New view **PIInvoicePeriodDetailView** at **/my-period//detail/** (**pi-period-detail**)
- Queries **SlurmJob.objects.filter(account=period.account, start_time*date*gte=period.start_date, start_time*date*lte=period.end_date)**
- Detail button added in **billing_report.html** ACTION column (between TXT and Del)
- No changes to existing PDF or TXT views

Job Usage Metrics (SlurmJob)

Per-job tracking is done via the **SlurmJob** model, populated by **collect_sacct**:

```
# Collect last 7 days of job data from sacct
python manage.py collect_sacct --current-week

# Collect current month
python manage.py collect_sacct --current-month

# Import from a pipe-delimited sacct dump file
python manage.py collect_sacct --from-file coldfront/templates/jobs_usage.csv
```

collect_sacct is also called automatically at the end of **run_slurm_sync()** (which runs daily via cron at 3:30 AM).

The Job Usage Metrics dashboard (**/billing/jobs/**) provides XDMoD-style analytics:

- Group by: User, Account, Partition, QOS, State, Department, School
- Sortable columns: CPU Hours, GPU Hours, Wall Hours, Jobs, Users, Avg CPUs, etc.
- Summary cards: Total CPU/GPU/Wall hours, job count, active users, avg wait time
- CSV export and raw job list view
- PI sees own jobs + jobs in their accounts; admin sees all

Submit Job UI

Any authenticated ColdFront user can submit a batch job at **/billing/submit/**.

What it does

- Paste or write a shell script (with **#SBATCH** directives)

- Choose a partition (loaded live from the active Slurm cluster)
- Choose a Slurm account (scoped to the user's allocations)
- Live cost estimate shown as you type based on **#SBATCH ntasks, cpus-per-task**, **time, gres=gpu**
- Scavenger / free partitions: submits directly with an "? No charge" badge no modal
- Billable partitions (e.g. **premium**): shows a cost confirmation modal (Resource / Hours / Rate / Charge table) before submitting user must click Agree & Submit to proceed

Account scoping

Role	Accounts shown
Staff / superuser	All `slurm_account_name` values from all allocations
PI	Only accounts from their own PI allocations
Regular user	Only accounts from allocations they are active members of

Billable partition detection

Partition billing is fully dynamic no hardcoded names. A partition is billable if it has an active **BillingRate** record in ColdFront (Admin -> Billing Rates). Adding a new billable partition via the admin automatically enables cost confirmation in the Submit Job UI.

Job Submission via SlurmClient

SlurmClient (in **coldfront/custom/plugins/arch_sync/slurm_client.py**) supports job submission and cancellation with two backends:

Backend	When used	Mechanism
docker-exec	`use_docker_exec=True` (Docker clusters)	`docker exec slurmctld sbatch --wrap ...`
REST API	bare-metal / UNIX socket	POST `/slurm/{version}/job/submit`

Submit a job

```
from coldfront.plugins.arch_sync.slurm_client import SlurmClient

client = SlurmClient.from_cluster(cluster)  # or construct directly
job_id = client.submit_job(
    script="#!/bin/bash\nsrun hostname",
    partition="cpu-dev",
    account="myaccount",
    job_name="test-job",
)
print(f"Submitted: {job_id}")
```

Cancel a job

```
client.cancel_job(job_id)
```

Management operations (existing)

Nodes, partitions, accounts, and job queries continue to use docker-exec or REST as before.

PySlurm (future option)

PySlurm provides Cython bindings to **libslurm.so** for direct Python-level Slurm calls. It is not currently installed in this stack because:

- PySlurm bindings must be built against the exact same Slurm version as the running daemons.
- ColdFront runs on **python:3.10-slim** (Debian); Slurm daemons run on Rocky Linux 9.6.
- Bridging these images requires either migrating ColdFront to Rocky Linux or running PySlurm inside the **slurmctld** container.

The current REST API + docker-exec **sbatch** approach covers all job submission needs without this complexity.

If PySlurm is needed in the future:

1. Pin Slurm version in **slurm/base/Dockerfile** (e.g. **slurm-25.11.2**).
2. Switch ColdFront base image to **rockylinux:9.6**.
3. Install matching **pyslurm** in the ColdFront container:

```
pip install pyslurm==22.5.9
```

4. Add a **PySlurm** backend to **SlurmClient**.

Pyxis / Enroot (Container Jobs)

Pyxis is a SPANK plugin that lets users run container images (via NVIDIA Enroot) as Slurm jobs. It is not currently installed in this stack documentation only.

What Pyxis provides

- **srun container-image=nvcr.io/nvidia/pytorch:24.01-py3 ...** syntax
- Rootless container execution via enroot (no Docker daemon required on compute nodes)
- Native GPU passthrough inside container jobs
- Works with **sbatch** and interactive **srun**

Adding Pyxis to CLOACK Stack

1. Install enroot on each compute node (**slurm/slurmd/Dockerfile**):

```
RUN dnf install -y enroot enroot+caps
```

2. Install pyxis SPANK plugin:

```
RUN dnf install -y pyxis
   # or build from source: https://github.com/NVIDIA/pyxis
```

3. Register the plugin in **slurm/conf/plugstack.conf**:

```
required /usr/lib64/slurm/pyxis.so
```

Use **required** so jobs fail cleanly if the plugin cannot load, rather than silently running without containers.

4. Restart slurmd on compute nodes after rebuilding the image.

Example job with Pyxis

```
srun \
  --partition=gpu \
  --account=myaccount \
  --container-image=nvcr.io/nvidia/pytorch:24.01-py3 \
  --container-mounts=/scratch:/scratch \
  python train.py
```

References

- Pyxis: https://github.com/NVIDIA/pyxis
- Enroot: https://github.com/NVIDIA/enroot
- Enroot + Pyxis setup guide: https://github.com/NVIDIA/pyxis/wiki/Setup

---

Slurm REST API Endpoints Used

ColdFront communicates with Slurm via **slurmrestd** (REST API). The **SlurmClient** class supports both Docker-exec and REST backends.

Action	Endpoint
List partitions (sinfo)	GET /slurm/v0.0.42/partitions
List nodes	GET /slurm/v0.0.42/nodes
List jobs (squeue)	GET /slurm/v0.0.42/jobs
Submit job	POST /slurm/v0.0.42/job/submit
Get job detail	GET /slurm/v0.0.42/job/{id}
Cancel job	DELETE /slurm/v0.0.42/job/{id}
Create account	POST /slurm/v0.0.42/accounts
Delete account	DELETE /slurm/v0.0.42/accounts/{name}
Create reservation	POST /slurm/v0.0.42/reservations
Delete reservation	DELETE /slurm/v0.0.42/reservation/{name}

Auth

Both JWT and munge supported chosen per SlurmCluster record. JWT token stored encrypted in the SlurmCluster model. UNIX socket auth uses **SO_PEERCRED** (peer-UID based).

Sync Pipeline

LDAPSync (ldap_sync.py)

Entry point: **run_ldap_sync(pi_username=None)**

Function	What it does
`sync_ldap_users()`	Create/update users in ou=People (routes to jhu or schmidt tree), sync admin/staff group, email, passwords
`sync_ldap_groups()`	Sync PI groups from ColdFront DB, create PI scratch directories
`sync_ldap_members()`	Add/remove members from PI LDAP groups, create home dirs, manage scratch symlinks

Tree routing: **ssci-*** username prefix ->** ou=schmidt**,** @jhu.edu **email ->** ou=jhu**, default ->** ou=jhu

SlurmSync (slurm_sync.py)

Entry point: **run_slurm_sync(pi_username=None)** bidirectional sync via sacctmgr/sreport CLI.

coldfront2slurm() PUSH:

- Ensure JHU root account (shares=20) and Schmidt root account (shares=80)
- Ensure PI account hierarchy under correct root (school -> dept -> PI -> project)
- Add active users to Slurm accounts, zero out inactive users (MaxJobs=0)
- Sync GrpTRESMins + fairshare from Core Usage Hours attribute
- Set weekly cap on billing TRES when **Weekly Cap ($)** is set

slurm2coldfront() PULL:

- Read actual CPU/billing hours per quarter via sreport
- Write back to AllocationAttribute: Core Usage (Hours)

How Syncs Are Triggered

Event	What runs
Allocation approved	LDAPSync (scratch dir + group) + SlurmSync (account + QOS)
Member added to project	LDAPSync (home dir + group) + SlurmSync (add user to account)
Password changed	LDAPSync (password hash to LDAP)
Daily cron (2 AM)	Full LDAP sync
Daily cron (3 AM)	Full Slurm sync

All async tasks dispatched via django-q with **TASK_HOOK** for admin LogEntry audit trail.

Cluster Model

Shared Cluster (Tier 1)

- Partitions are pre-defined by admins (not created per PI request)
- PI gets allocation approved -> Slurm account created under JHU or Schmidt root
- Account gets access to partitions based on QOS (scavenger for default, normal+premium for paid)
- Partition is chosen at job submission time (not at allocation time)

Reserved Tier (Tier 2 -- optional, future)

- PI requests reservation -> specifies nodes + time window
- Admin approves + optional budget/payment gate
- Plugin creates Slurm reservation via REST API
- Priority QOS applied (bypasses fair-share queue)
- On expiry: nodes return to shared pool automatically

Troubleshooting

SSSD not resolving users after docker restart

Symptom: SSH to login node fails with **Invalid user** or **id** returns "no such user", but the user exists in LDAP.

Cause: SSSD is started as a background process by the entrypoint script, not by a service manager. When you run **docker restart login01**, the container PID 1 (entrypoint) restarts and re-launches SSSD. However, if you restart the container without compose (e.g., **docker restart** directly), the **LDAP_ADMIN_PASS** env var may not be set, causing the SSSD startup block to be skipped.

Fix:

```
# Option 1: Restart via start.sh (recommended -- preserves env vars from compose)
./start.sh slurm restart

# Option 2: Manual SSSD start inside the container
docker exec login01 sssd -D

# Option 3: Recreate via compose (guaranteed clean state)
./start.sh slurm stop && ./start.sh slurm start
```

Prevention: Always use **./start.sh slurm restart** or **docker compose up** instead of **docker restart** for Slurm containers that depend on SSSD.

slurmrestd returns 404 on API calls

Symptom: ColdFront cluster detail page shows "Cannot reach slurmrestd: 404".

Cause: The cluster's API version (**api_version** field) does not match what slurmrestd serves. Slurm 25.11 serves **v0.0.41** and **v0.0.42**, not **v0.0.40**.

Fix: Update the cluster API version in Django Admin > Slurm Clusters > edit the cluster > set API Version to **v0.0.42**.

slurmrestd returns 502 but data is valid

Symptom: Partition queries return HTTP 502 with error "Slurmdb query failed: Header lengths are longer than data received" but the JSON response body contains valid partition data.

Cause: Known Slurm 22.05 issue the TRES metadata query to slurmdbd fails, but partition data is still returned. The **slurm_client.py** handles this gracefully by accepting valid JSON responses even on non-2xx status codes.

No action needed this is handled automatically.

Nodes stuck in "down" state after container recreate

Symptom: **sinfo** shows nodes as **down** after recreating slurmd containers.

Cause: slurmctld marks nodes as down when slurmd disconnects during container recreation.

Fix: The slurmd entrypoint includes a background **scontrol update state=resume** that runs automatically. If nodes remain down:

```
docker exec slurmctld scontrol update nodename=cn-01 state=resume reason="container recreated"
```

slurmctld crashes with "No Assoc usage file" on fresh cluster (Slurm 25.11)

Symptom: slurmctld exits with **fatal: No Assoc usage file (/var/spool/slurm/slurmctld/assoc_usage) to recover** on a brand-new cluster with no previous state.

Cause: Slurm 25.11 introduced mandatory association usage state recovery. On a fresh cluster the **assoc_usage** binary file does not exist in **StateSaveLocation**. Unlike previous versions, 25.11 treats this as a fatal error instead of creating the file on first start. An empty placeholder file also fails because slurmctld validates the binary header (requires version 10496-11264). See SchedMD Bug 20125.

Fix: The slurmctld entrypoint uses the **-i** flag ("ignore state errors") on startup. This tells slurmctld to skip incompatible/missing state files and start fresh. Once running, slurmctld creates valid state files in **StateSaveLocation** so subsequent restarts work normally without **-i**.

```
# In slurmctld/entrypoint.sh:
exec slurmctld -D -v -i
```

Note: The **-i** flag discards any previous job/association usage data. This is safe for Docker dev clusters (no real usage to lose). For production bare-metal upgrades, use **scontrol reconfigure** after migrating state files from the previous version.

AllowQos on partitions causes fatal on fresh cluster

Symptom: slurmctld exits with **fatal: Partition premium has an invalid AllowQOS (normal,premium)** on first start.

Cause: The generated **slurm.conf** includes **AllowQos=scavenger,normal,premium** on partition lines, but QOS records do not exist yet in slurmdbd (fresh cluster). slurmctld validates QOS names at startup.

Fix: The **start.sh slurm start** sequence runs **sync_slurm** after slurmctld starts, which creates the QOS entries. On the very first boot, partitions start without **AllowQos**; the next **scontrol reconfigure** picks up the QOS constraints after sync creates them.

Multi-Cluster Federation (Roadmap)

Slurm supports two multi-cluster architectures. Federation is recommended for JHU.

Federation -- multiple slurmctld, shared slurmdbd

```
+-------------+
                    |  slurmdbd   | <- 1 unique (shared accounting DB)
                    |  slurmrestd | <- 1 unique (REST API for all clusters)
                    L------+------+
              +------------+------------+
              ?            ?            ?
        +----------+ +----------+ +----------+
        |slurmctld | |slurmctld | |slurmctld |
        |  "arch"  | |"discovery"| |  "dev"  |
        L----+-----+ L----+-----+ L----+-----+
             ?            ?            ?
        +--------+  +----------+  +--------+
        |cn-01..N|  |gpu-01..N |  |dev-01  |
        L--------+  L----------+  L--------+
              ?            ?            ?
              L------------+------------+
                     +-----+-----+
                     |  login01  | <- shared login node
                     L-----------+
```

Benefits:

- Unified accounting database billing across all clusters
- **sacct -M arch**, **sacct -M discovery** same commands, **-M** selects cluster
- **sbalance**, **slurmtree**, **slurmqos** work with **-M** flag
- 1 slurmrestd serves all clusters via **/slurmdb/v0.0.42/jobs?cluster=X**
- Shared login node users submit with **sbatch -M discovery**
- Same users/accounts across clusters (associations per cluster)

Independent clusters -- separate slurmdbd per cluster

```
+--------------+    +--------------+
|  slurmdbd-1  |    |  slurmdbd-2  |
|  restd-1     |    |  restd-2     |
L------+-------+    L------+-------+
       ?                   ?
  arch + disc             dev
```

Each slurmdbd manages N clusters. Each slurmrestd connects to its own slurmdbd. More complex but isolates failures.

ColdFront integration

The **SlurmCluster** model already supports both architectures:

```
# Federation: same slurmrestd for all
SlurmCluster(name="arch",      slurmrestd_url="http://slurmrestd:6820")
SlurmCluster(name="discovery", slurmrestd_url="http://slurmrestd:6820")

# Independent: separate endpoints
SlurmCluster(name="arch",      slurmrestd_url="http://restd-1:6820")
SlurmCluster(name="dev",       slurmrestd_url="http://restd-2:6820")
```

Current code uses **-M cluster_name** (auto-injected by **_slurm_command_docker()**) which works with Federation out of the box. Future work:

- Resource <-> SlurmCluster link: Allocation -> Resource -> SlurmCluster for per-cluster routing
- Per-cluster sync: **slurm_sync** and **collect_sacct** iterate all active clusters
- Multi-org on single cluster: Multiple organizations (JHU, Schmidt) share one cluster with separate **root_account** values

Slurm Restart Guide

When SLURM services show warning or error status in the Service Health dashboard, you can use the UI "Restart" buttons to restart them automatically, or run manual commands as shown below.

SLURM Services Overview

- slurmdbd (SLURM Database Daemon) Accounting and job history database
- slurmctld (SLURM Controller Daemon) Primary cluster controller
- slurmrestd (SLURM REST API Daemon) REST API for job submission and queries
- cn-01, cn-02, cn-03 Compute node daemons (slurmd)
- login01 Login node daemon (slurmd)

Restart Order (Important)

When restarting SLURM services, follow this order to avoid dependency issues:

1. slurmdbd Start the database daemon first
2. slurmctld Then the controller daemon
3. slurmrestd Then the REST API
4. Compute nodes Finally the compute node daemons
5. Login nodes And login node daemons

Quick Restart Commands

Using Docker Compose (Recommended)

The project uses **docker-compose-slurm-arch-dev.yml** to manage SLURM services.

Restart individual services:

```
# Restart slurmdbd
docker compose -f docker-compose-slurm-arch-dev.yml restart slurmdbd

# Restart slurmctld
docker compose -f docker-compose-slurm-arch-dev.yml restart slurmctld

# Restart slurmrestd
docker compose -f docker-compose-slurm-arch-dev.yml restart slurmrestd

# Restart a compute node
docker compose -f docker-compose-slurm-arch-dev.yml restart cn-01

# Restart login node
docker compose -f docker-compose-slurm-arch-dev.yml restart login01
```

Restart all SLURM services (proper order):

```
docker compose -f docker-compose-slurm-arch-dev.yml restart slurmdbd
sleep 10
docker compose -f docker-compose-slurm-arch-dev.yml restart slurmctld
sleep 5
docker compose -f docker-compose-slurm-arch-dev.yml restart slurmrestd
docker compose -f docker-compose-slurm-arch-dev.yml restart cn-01 cn-02 cn-03 login01
```

Using Docker Directly

Check service status:

```
docker ps | grep slurm
```

Restart a specific container:

```
# Get the container ID first
docker ps -f name=slurmdbd --format "table {{.ID}}\t{{.Names}}\t{{.Status}}"

# Then restart it
docker restart <container_id>
```

UI-Based Restart

1. Go to Admin Menu -> Service Health (or **/user/service-health/**)
2. Look for SLURM services showing ?? Warn or ? Error status
3. Click the Restart button to restart the service
4. Monitor the Restart Results panel for status and logs

Troubleshooting

Services Won't Start

Check logs:

```
docker compose -f docker-compose-slurm-arch-dev.yml logs slurmdbd
docker compose -f docker-compose-slurm-arch-dev.yml logs slurmctld
```

Check dependencies:

- **slurmdbd** requires the MariaDB database to be healthy
- **slurmctld** requires **slurmdbd** to be healthy first
- Compute nodes require **slurmctld** to be running

Test Service Health

```
# Test slurmrestd API connectivity
curl -X GET http://localhost:6820/slurm/v0.0.36/ping

# Check slurmdbd database
docker exec slurmdbd mysqladmin -u slurm -p<password> ping

# Check slurmctld status
docker exec slurmctld sinfo
```

Health Check Indicators

The Service Health page shows:

- ? OK Service is running and healthy
- ? Warn Service is reachable but may have issues
- ? Error Service is unreachable or not responding

Related Configuration

- Docker Compose config: **docker-compose-slurm-arch-dev.yml**
- SLURM config: **slurm/conf/slurm.conf**
- SLURM DB config: **slurm/conf/slurmdbd.conf**

For more details on SLURM administration, see the official SLURM documentation.

OpenLDAP

Centralized POSIX-compatible identity directory for JHU and Schmidt clusters. Stores users, groups, and membership. Runs SSSD for local NSS/PAM caching.

Build

```
docker build -t arch/openldap openldap/
```

Image: **osixia/openldap:1.5.0** (multi-platform: amd64 + arm64).

Directory Structure

```
openldap/
|-- Dockerfile
|-- entrypoint-wrapper.sh    -- Runs init scripts before osixia entrypoint
|-- healthcheck-wrapper.sh   -- Runs replace_ldif.sh once when LDAP becomes healthy
|-- sssd.conf                -- SSSD config (LDAP caching, NSS/PAM)
|-- ldap.conf                -- OpenLDAP client defaults
|-- certs/                   -- TLS certificates
|-- certs.sh                 -- Certificate generation
L-- local/
    L-- bin/                 -- Custom scripts (LDIF generation, user/group management)
```

LDAP Schema

- Base DN: **dc=arch,dc=cluster**
- Admin DN: **cn=admin,dc=arch,dc=cluster**
- JHU users: **ou=People,ou=jhu,dc=arch,dc=cluster**
- JHU groups: **ou=Groups,ou=jhu,dc=arch,dc=cluster**
- Schmidt users: **ou=People,ou=schmidt,dc=arch,dc=cluster**
- Schmidt groups: **ou=Groups,ou=schmidt,dc=arch,dc=cluster**
- Schema: RFC2307bis (posixAccount/posixGroup)

Environment Variables

Variable	Purpose
`LDAP_ADMIN_PASSWORD`	Admin bind password
`LDAP_ORGANISATION`	Organization name
`LDAP_DOMAIN`	Domain (e.g. `arch.cluster`)

Port

- 389 (LDAP)

Debugging

```
# Search all users
docker exec openldap ldapsearch -x -D "cn=admin,dc=arch,dc=cluster" -w $LDAP_ADMIN_PASS -b "dc=arch,dc=clus...

# Search groups
docker exec openldap ldapsearch -x -D "cn=admin,dc=arch,dc=cluster" -w $LDAP_ADMIN_PASS -b "dc=arch,dc=clus...
```

Keycloak

OAuth 2.0/OIDC identity broker. Manages two realms (JHU and Schmidt) with LDAP federation and optional Azure Entra ID integration. Provides SSO for ColdFront.

Build

```
docker build -t arch/keycloak keycloak/
```

Image: **quay.io/keycloak/keycloak:latest** with **jq** for JSON processing.

Directory Structure

```
keycloak/
|-- Dockerfile
|-- configure-keycloak.sh       -- Auto-configures realms, LDAP providers, clients
|-- healthcheck-wrapper.sh      -- Healthcheck + trigger configure on first ready
|-- ldap-providers/
|   |-- jhu-ldap-provider.json      -- JHU LDAP federation config
|   L-- schmidt-ldap-provider.json  -- Schmidt LDAP federation config
|-- themes/
|   |-- arch-jhu/         -- JHU login theme
|   L-- arch-schmidt/     -- Schmidt login theme
L-- certs/                -- TLS certificates
```

Realms & LDAP Federation

Realm	LDAP Users Base	LDAP Groups Base	Theme
`jhu`	`ou=People,ou=jhu`	`ou=Groups,ou=jhu`	`arch-jhu`
`schmidt`	`ou=People,ou=schmidt`	`ou=Groups,ou=schmidt`	`arch-schmidt`

Both federations: daily sync, connection pooling, WRITABLE mode.

Environment Variables

Variable	Purpose
`KEYCLOAK_ADMIN`	Admin username
`KEYCLOAK_ADMIN_PASSWORD`	Admin password
`LDAP_ADMIN_PASS`	LDAP bind password
`JHU_ENTRA_ENABLED`	`1` = enable Azure Entra ID for JHU realm
`JHU_ENTRA_CLIENT_ID`	Entra application ID
`JHU_ENTRA_CLIENT_SECRET`	Entra client secret
`KEYCLOAK_JHU_REALM`	JHU realm name (default: `jhu`)
`KEYCLOAK_SCHMIDT_REALM`	Schmidt realm name (default: `schmidt`)

Port

- 8443 (HTTPS)

Output

Auto-exports Django OIDC client secrets to **/shared/coldfront.env** for downstream ColdFront consumption.

PostgreSQL

Primary data persistence layer. Initializes and maintains three databases for the CLOACK stack.

Build

```
docker build -t arch/postgres postgres/
```

Image: **postgres:16**.

Databases

Database	Owner	Purpose
`keycloak`	`keycloak`	Keycloak realms, clients, providers
`coldfront`	`coldfront`	ColdFront Django application data
`slurm_jobs`	`coldfront`	High-volume Slurm job metrics (separate DB for performance)

Created idempotently by **init-databases.sh** via Docker's **/docker-entrypoint-initdb.d/** mechanism.

Directory Structure

```
postgres/
|-- Dockerfile
L-- init-databases.sh    -- Idempotent SQL initialization (3 DBs + 2 users)
```

Environment Variables

Variable	Purpose
`POSTGRES_USER`	Admin user (default: `postgres`)
`POSTGRES_PASSWORD`	Admin password
`KEYCLOAK_DB_NAME`	Keycloak DB name
`KEYCLOAK_DB_USER`	Keycloak DB user
`KEYCLOAK_DB_PASSWORD`	Keycloak DB password
`COLDFRONT_DB_NAME`	ColdFront DB name
`COLDFRONT_DB_USER`	ColdFront DB user
`COLDFRONT_DB_PASSWORD`	ColdFront DB password

Port

- 5432 (PostgreSQL)

Healthcheck

pg_isready (10-second interval, 10 retries).

Base Configuration

Centralized configuration shared across all services.

Files

File	Purpose
`base.config`	Static defaults: UID/GID ranges, paths, hostnames, domain names
`bin/`	Shared utility scripts

base.config

Sourced by multiple services (postgres, keycloak, openldap) for consistent UID/GID ranges and default credentials. Key values:

- SLURM_UID / MUNGE_UID System account UIDs (must match across all Slurm nodes)
- LDAP_DOMAIN, LDAP_BASE_DN LDAP directory settings
- Default passwords Used only for initial dev bootstrap

Architecture Overview

ARCH Portal

- Home
- README
- CLOACK
- Architecture
- Cluster Models
- Help

CLOACK STACK DEV v0.3

ColdFront · LDAP · OIDC · Accounts · Clusters · Keycloak

Django 4.1 | PostgreSQL 16 | OpenLDAP | Keycloak 24 | Slurm 25.11 | Docker Compose | Rocky Linux 9 | SSSD | Munge | REST API | Django-Q | OIDC

VERSION SUMMARY — DEV V0.3 ▼ CLICK TO EXPAND

USERS

- 🛠 Admin / Staff — Manage · Approve · Deploy
- 📚 PI — Projects · Allocations · Billing
- 👤 User — Login · Submit Jobs · View Metrics

↓

CORE SERVICES (DOCKER COMPOSE)

- ColdFront (Portal) — Django + arch_sync :8443
- Keycloak (OIDC) — JHU + Schmidt realms :8080
- OpenLDAP (Directory) — ou=jhu · ou=schmidt :636
- PostgreSQL (Database) — 3 databases :5432

↓

SLURM CLUSTER

- slurmctld — Controller :6817
- slurmdbd — Accounting :6819
- slurmrestd — REST API :6820
- Compute Nodes — cn-01..N · SSSD Munge + SPANK
- Login Node — SSH · sbatch cli_filter.lua

↓

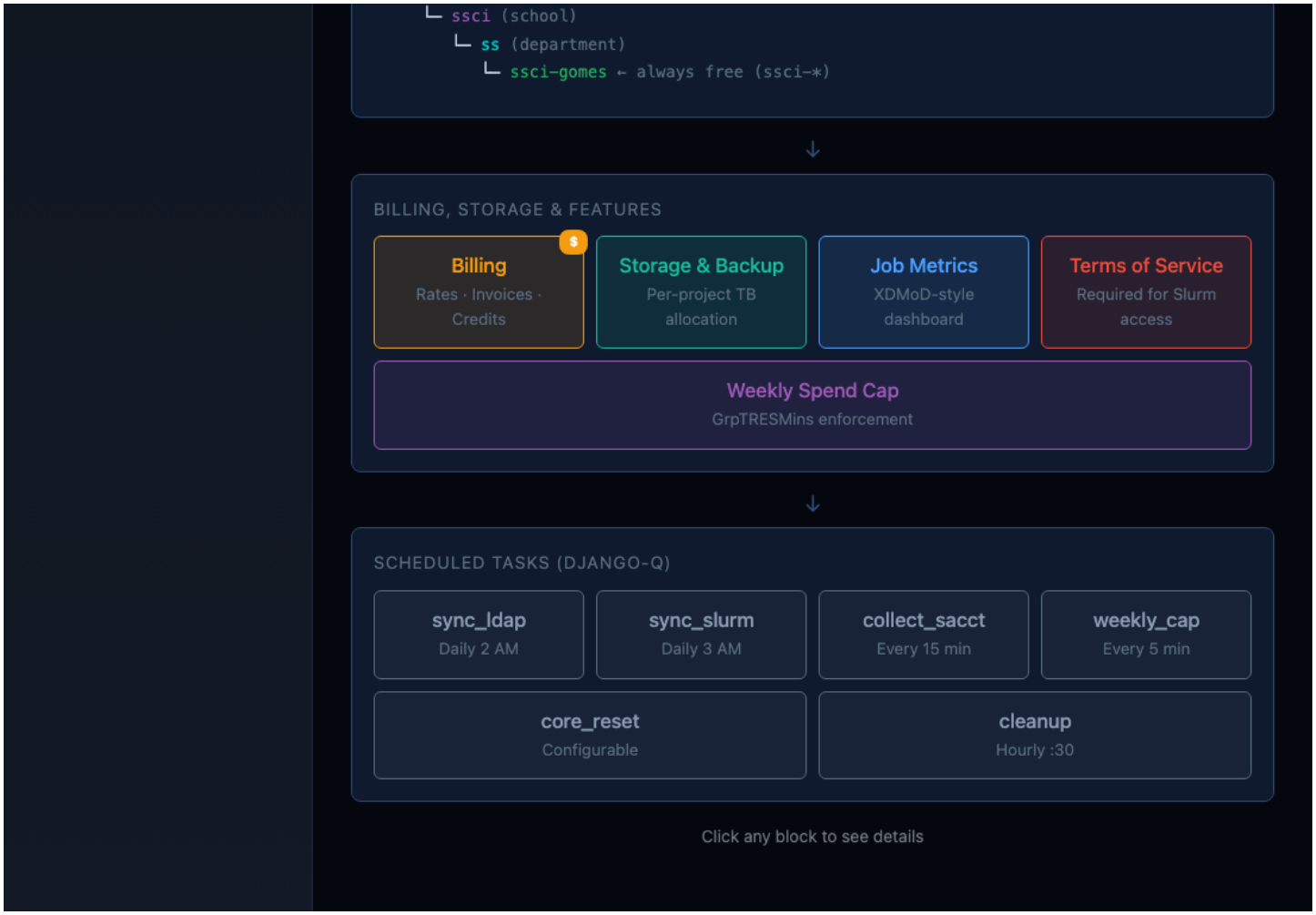
PARTITIONS & ALLOCATION TIERS

- scavenger (FREE) — Default · Preemptible
- premium (PAID) — High-priority
- gpu-a100 (GPU) — TRESBillingWeights
- dev — Testing

↓

SLURM ACCOUNT HIERARCHY

```
root
 ├─ jhu [20] (organization)
 │  ├─ it (school)
 │  │  ├─ arch (department)
 │  │  │  ├─ rdesouz4 ← default (scavenger)
 │  │  │  └─ rdesouz4_proj1 ← paid (premium)
 └─ schmidt [80] (organization)
```


└ ssci (school)
└ ss (department)
└ ssci-gomes ← always free (ssci-*)
BILLING, STORAGE & FEATURES
$
Billing
Rates · Invoices · Credits
Storage & Backup
Per-project TB allocation
Job Metrics
XDMoD-style dashboard
Terms of Service
Required for Slurm access
Weekly Spend Cap
GrpTRESMins enforcement
SCHEDULED TASKS (DJANGO-Q)
sync_ldap
Daily 2 AM
sync_slurm
Daily 3 AM
collect_sacct
Every 15 min
weekly_cap
Every 5 min
core_reset
Configurable
cleanup
Hourly :30
Click any block to see details